CS2013: Programación III

Teoría: Algoritmo de Dijkstra y Kruskal

José Chávez

Agenda

- El camino más corto
- Algoritmo de Dijkstra
- Árboles de Expansión
- Algoritmo de Kruskal

1. El Camino Más Corto

El Camino Más Corto

El Algoritmo de **Dijkstra** sirve para calcular el **camino más corto** (con menos costo) desde un nodo a todos los demás nodos de un grafo **dirigido/no-dirigido** y **ponderado** con pesos **no negativos**.

El Camino Más Corto

El Algoritmo de Dijkstra sirve para calcular el camino más corto (con menos costo) desde un nodo a todos los demás nodos de un grafo dirigido/no-dirigido y ponderado con pesos no negativos.

Para esto el algoritmo utiliza una estrategia de exploración de nodos de manera **iterativa**. Comienza desde el **nodo de origen** y va explorando los **nodos adyacentes**, actualizando continuamente las distancias mínimas de los nodos explorados hasta el momento.

El Camino Más Corto

El Algoritmo de Dijkstra sirve para calcular el camino más corto (con menos costo) desde un nodo a todos los demás nodos de un grafo dirigido/no-dirigido y ponderado con pesos no negativos.

Para esto el algoritmo utiliza una estrategia de exploración de nodos de manera iterativa. Comienza desde el nodo de origen y va explorando los nodos adyacentes, actualizando continuamente las distancias mínimas de los nodos explorados hasta el momento.

Disjkstra es un ***algoritmo voraz***, ya que el algoritmo toma decisiones locales óptimas en cada etapa, con la esperanza de llegar a una solución global óptima al final.

Aplicaciones

Aplicaciones

- **Enrutamiento de redes:** En protocolos de enrutamiento para encontrar el camino más corto entre dos nodos en una red.

Aplicaciones

- **Enrutamiento de redes:** En protocolos de enrutamiento para encontrar el camino más corto entre dos nodos en una red.
- **Sistemas de navegación:** En aplicaciones de navegación para encontrar la ruta más rápida o más corta entre dos ubicaciones, como en sistemas de GPS.

Aplicaciones

- **Enrutamiento de redes:** En protocolos de enrutamiento para encontrar el camino más corto entre dos nodos en una red.
- **Sistemas de navegación:** En aplicaciones de navegación para encontrar la ruta más rápida o más corta entre dos ubicaciones, como en sistemas de GPS.
- **Optimización de rutas de transporte:** En la distribución de mercancías o la programación de horarios de transporte público.

Aplicaciones

- **Enrutamiento de redes:** En protocolos de enrutamiento para encontrar el camino más corto entre dos nodos en una red.
- **Sistemas de navegación:** En aplicaciones de navegación para encontrar la ruta más rápida o más corta entre dos ubicaciones, como en sistemas de GPS.
- **Optimización de rutas de transporte:** En la distribución de mercancías o la programación de horarios de transporte público.
- **Redes de transporte público:** Para determinar los tiempos de llegada y las rutas óptimas en redes de transporte público, como en sistemas de metro o autobús.



2. Algoritmo de Dijkstra

Preliminares

Preliminares

La distancia del nodo origen a los otros nodos:

- La distancia del nodo origen a sí mismo es 0.
- La distancia del nodo origen a los demás se inicializa con infinito.

Preliminares

La distancia de un nodo a los otros nodos:

- La distancia de un nodo a sí mismo es 0.
- La distancia de un nodo a los demás se inicializa con infinito.

Complejidad:

- El algoritmo tiene una complejidad de $O((V+E)logV)$, donde V es el número de nodos/vértices y E el número de aristas.

Algoritmo de Dijkstra

Algoritmo de Dijkstra

1. Seleccionar un nodo **u** como el **origen** y marcarlo como **‘no-visitado’.**

Algoritmo de Dijkstra

1. Seleccionar un nodo **u** como el **origen** y marcarlo como **‘no-visitado’.**
2. Se inicializan todas las distancias desde el **origen** a todos los nodos.

Algoritmo de Dijkstra

1. Seleccionar un nodo **u** como el **origen** y marcarlo como **‘no-visitado’.**
2. Se inicializan todas las distancias desde el **origen** a todos los nodos.
3. Mientras existan nodos no visitados:

Algoritmo de Dijkstra

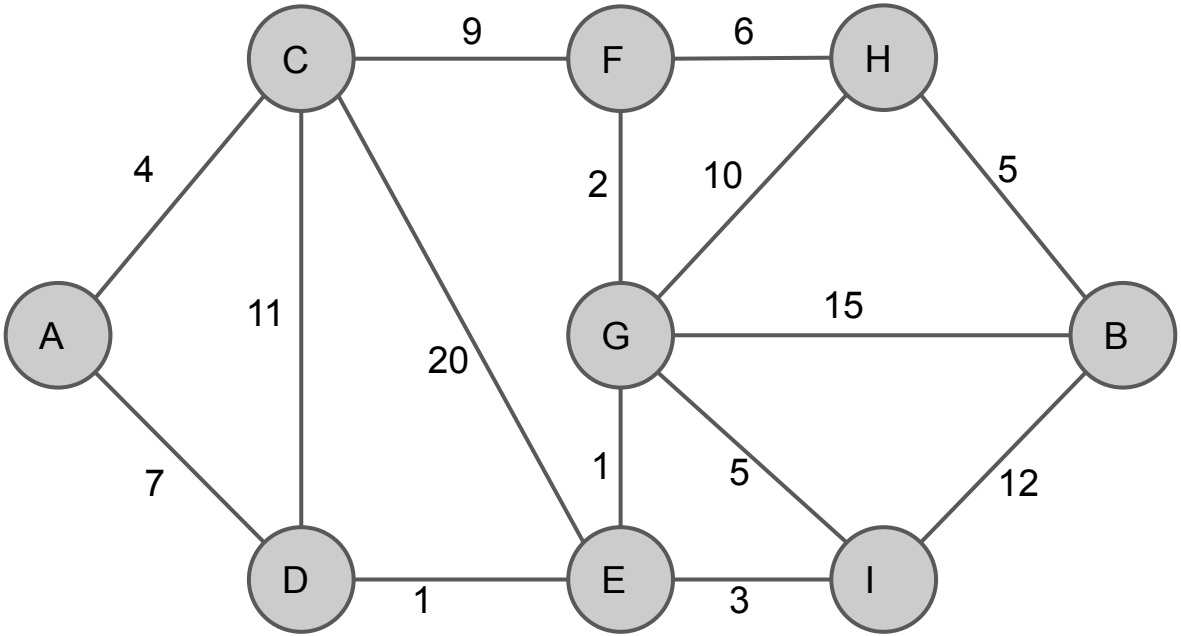
1. Seleccionar un nodo **u** como el **origen** y marcarlo como **‘no-visitado’.**
2. Se inicializan todas las distancias desde el **origen** a todos los nodos.
3. Mientras existan nodos no visitados:
   3.1. Seleccionar un nodo **v** , **no-visitado,** con la distancia mínima.
   3.2. Marcar el nodo **v** como **visitado**.

Algoritmo de Dijkstra

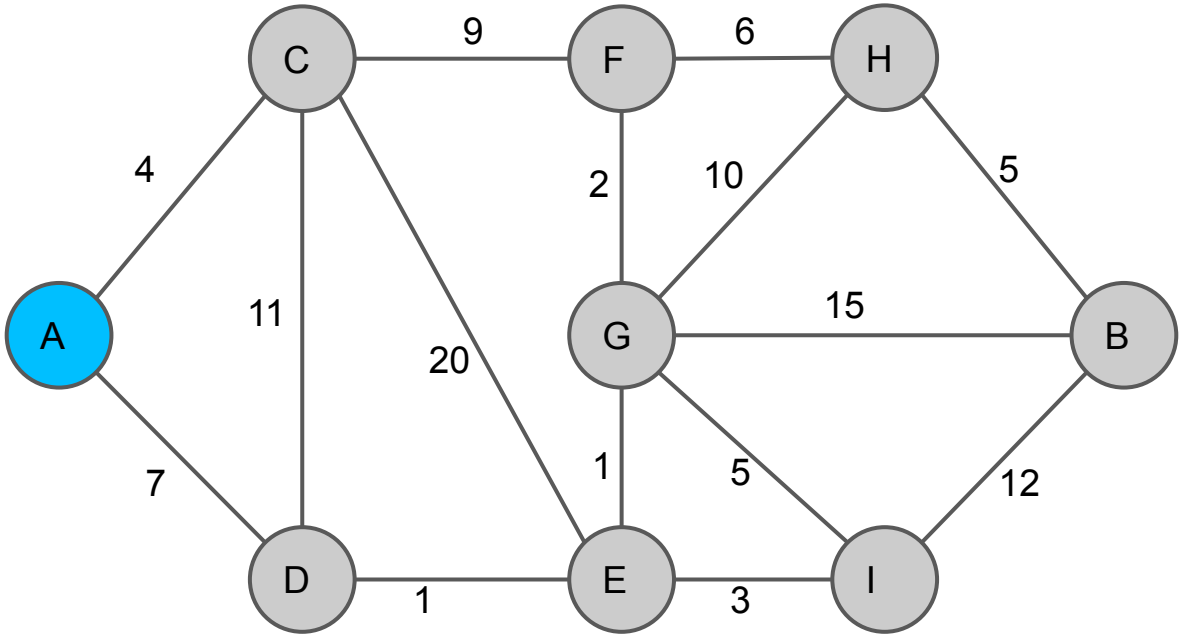
1. Seleccionar un nodo **u** como el **origen** y marcarlo como **‘no-visitado’.**
2. Se inicializan todas las distancias desde el **origen** a todos los nodos.
3. Mientras existan nodos no visitados:
   3.1. Seleccionar un nodo **v**, **no-visitado,** con la distancia mínima.
   3.2. Marcar el nodo **v** como **visitado**.
   3.3. Se calcula la distancia a un nodo adyacente **vi** con la siguiente fórmula:

```
d(vi) = MIN(d(v) + peso(v,vi), d(vi))
```

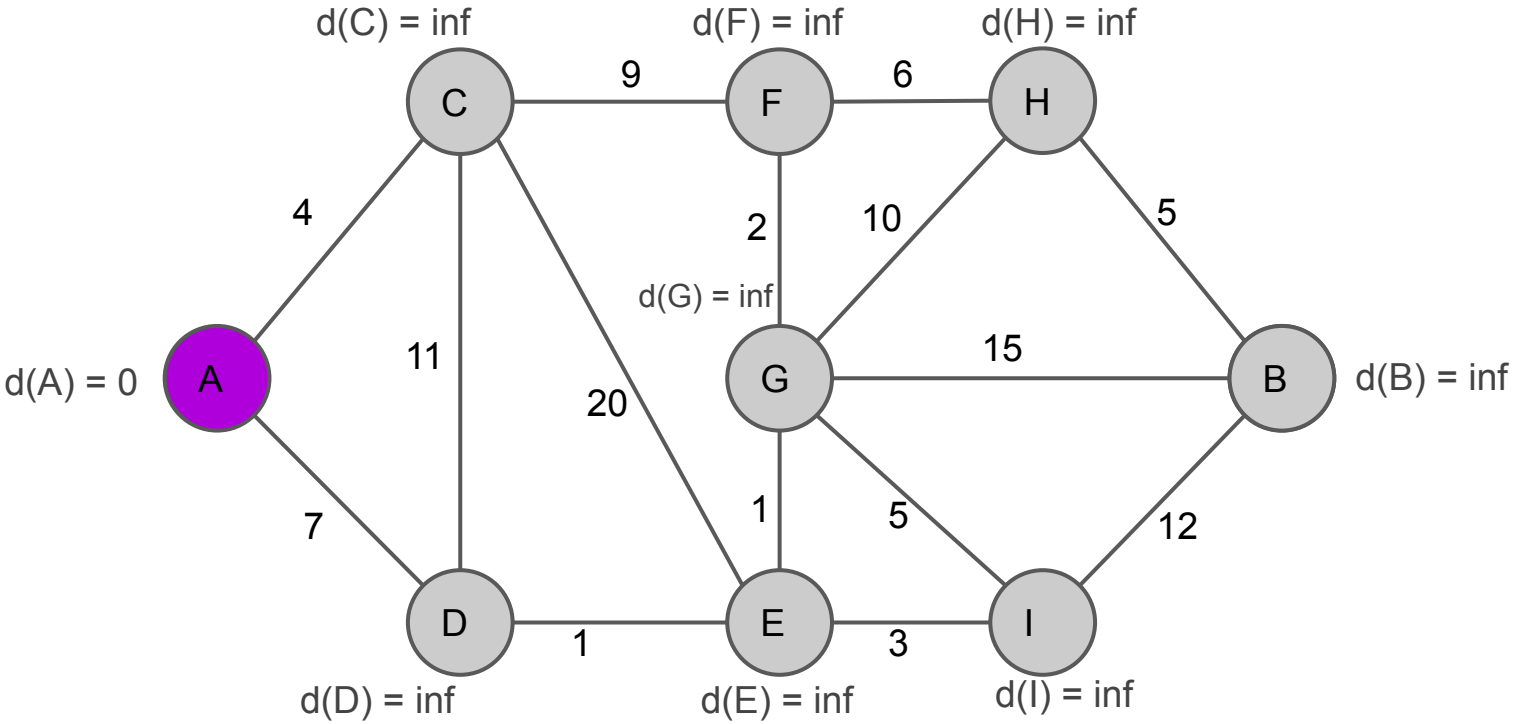
Ejemplo



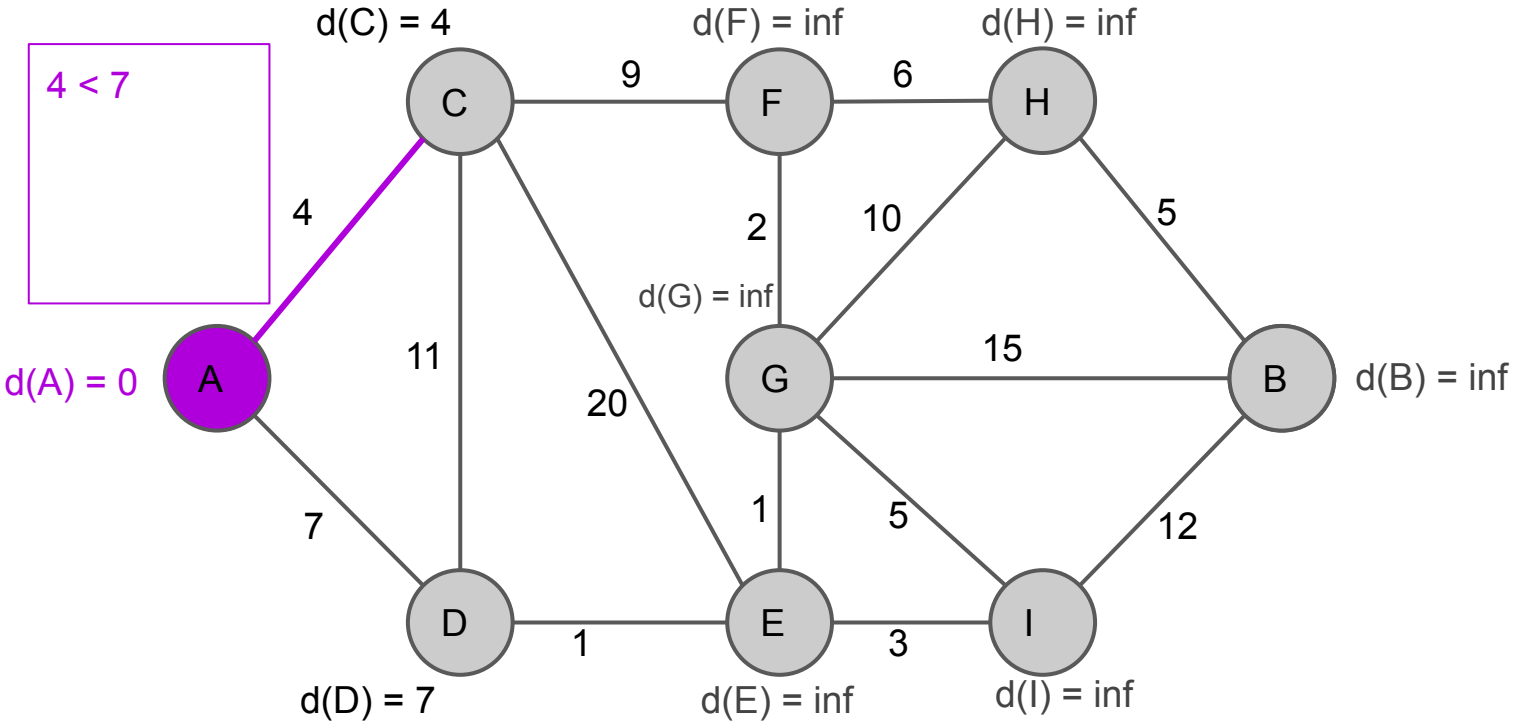
Ejemplo



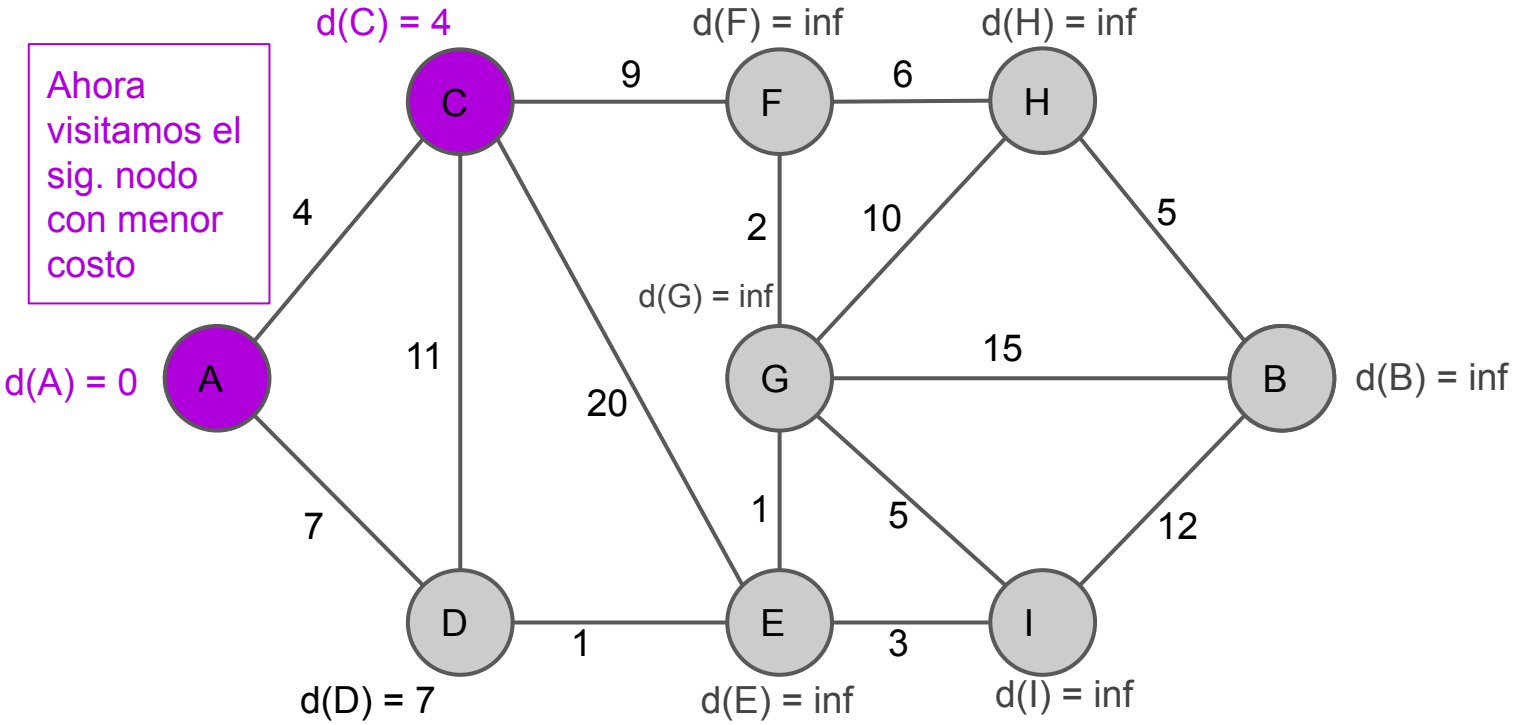
Ejemplo


d(C) = inf
d(F) = inf
d(H) = inf
C
F
H
9
6
4
2
10
5
d(G) = inf
d(A) = 0
A
11
G
15
B
d(B) = inf
20
1
5
12
7
D
E
I
1
3
d(D) = inf
d(E) = inf
d(I) = inf

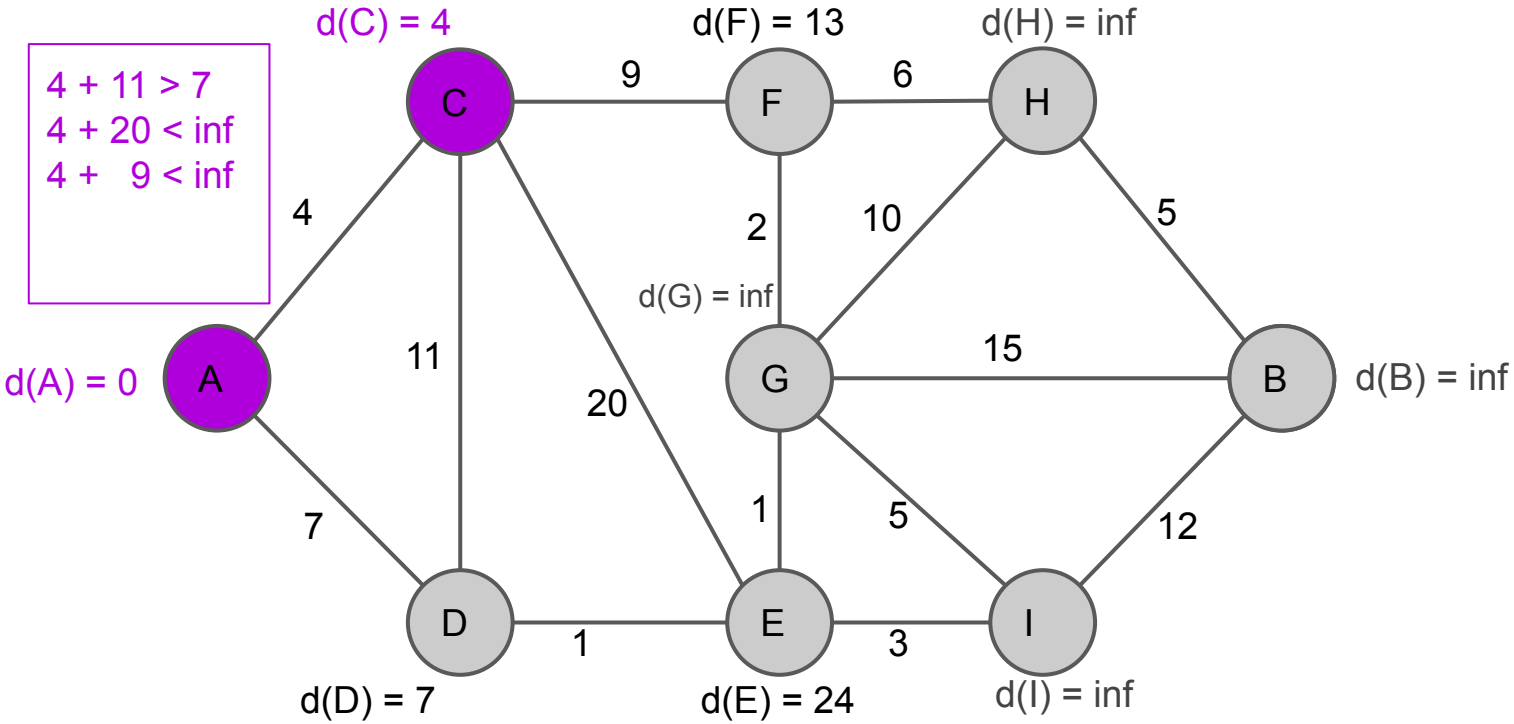
Ejemplo


4 < 7
d(C) = 4
d(F) = inf
d(H) = inf
C
F
H
9
6
4
2
10
5
d(G) = inf
d(A) = 0
A
11
G
15
B
d(B) = inf
20
7
1
5
12
D
E
I
1
3
d(D) = 7
d(E) = inf
d(I) = inf

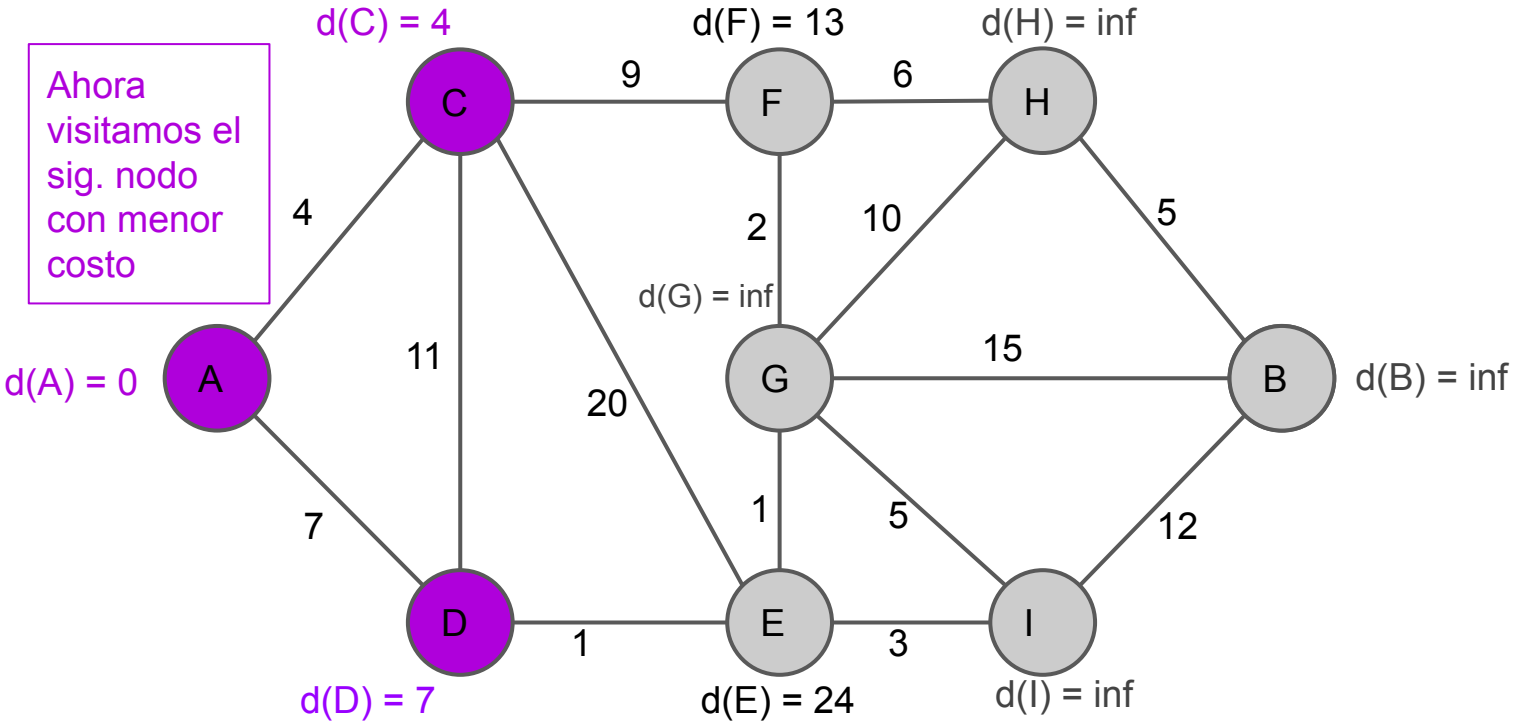
Ejemplo


Ahora visitamos el sig. nodo con menor costo
d(C) = 4
d(F) = inf
d(H) = inf
C
F
H
9
6
4
2
10
5
d(G) = inf
d(A) = 0
A
11
G
15
B
d(B) = inf
20
7
1
5
12
D
E
I
1
3
d(D) = 7
d(E) = inf
d(I) = inf

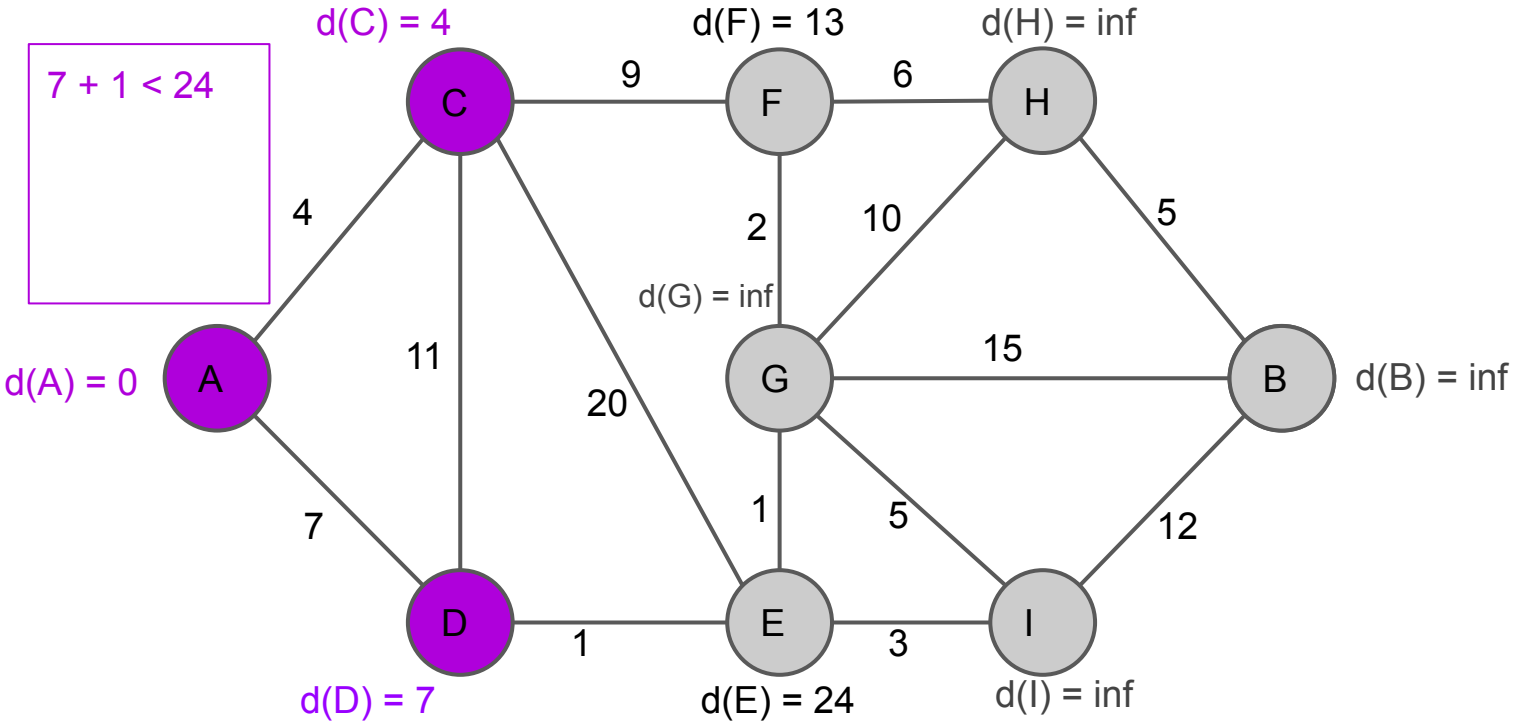
Ejemplo


d(C) = 4
d(F) = 13
d(H) = inf
4 + 11 > 7
4 + 20 < inf
4 + 9 < inf
C
F
H
9
6
4
2
10
5
d(G) = inf
d(A) = 0
A
11
G
15
B
d(B) = inf
20
7
1
5
12
D
E
I
1
3
d(D) = 7
d(E) = 24
d(I) = inf

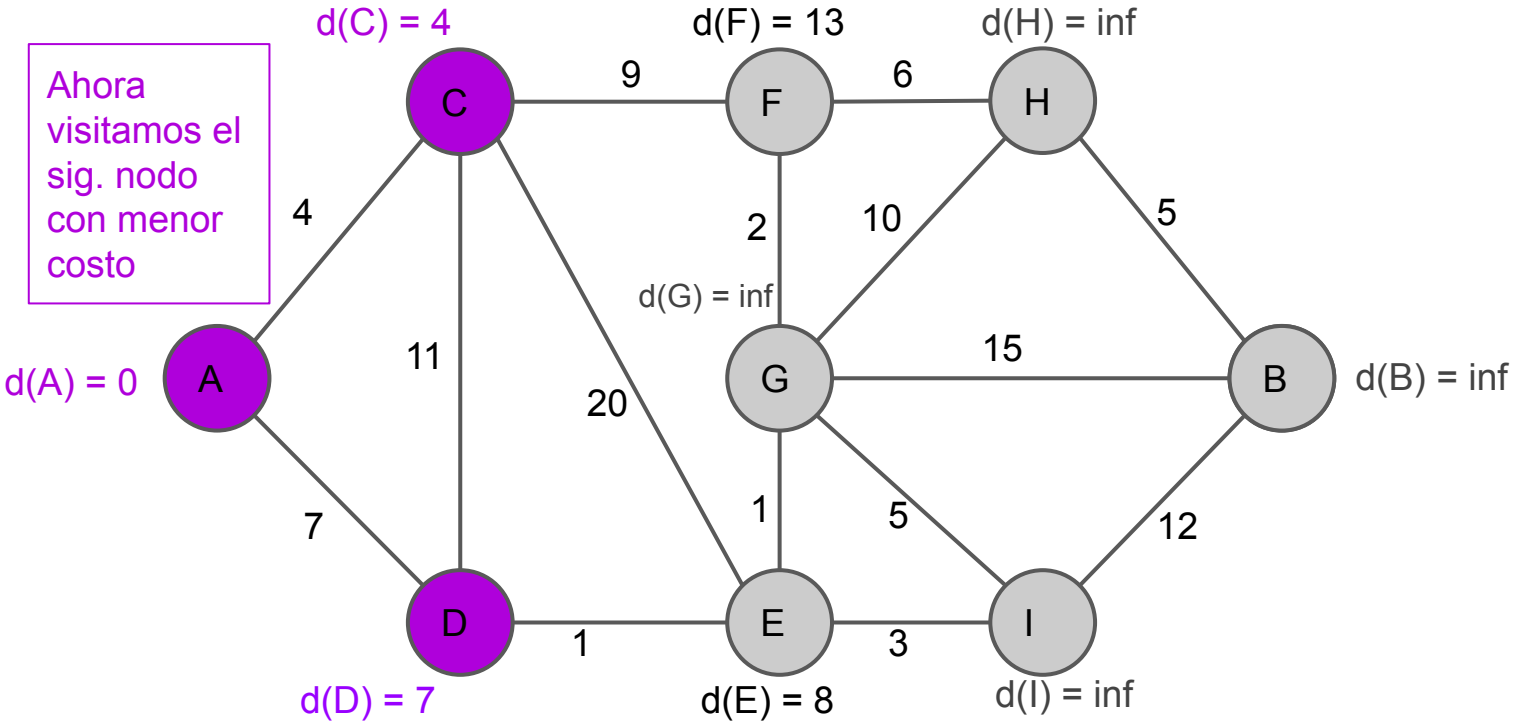
Ejemplo


Ahora visitamos el sig. nodo con menor costo
d(C) = 4
d(F) = 13
d(H) = inf
C
F
H
9
6
4
2
10
5
d(G) = inf
d(A) = 0
A
11
G
15
B
d(B) = inf
20
7
1
5
12
D
E
I
1
3
d(D) = 7
d(E) = 24
d(I) = inf

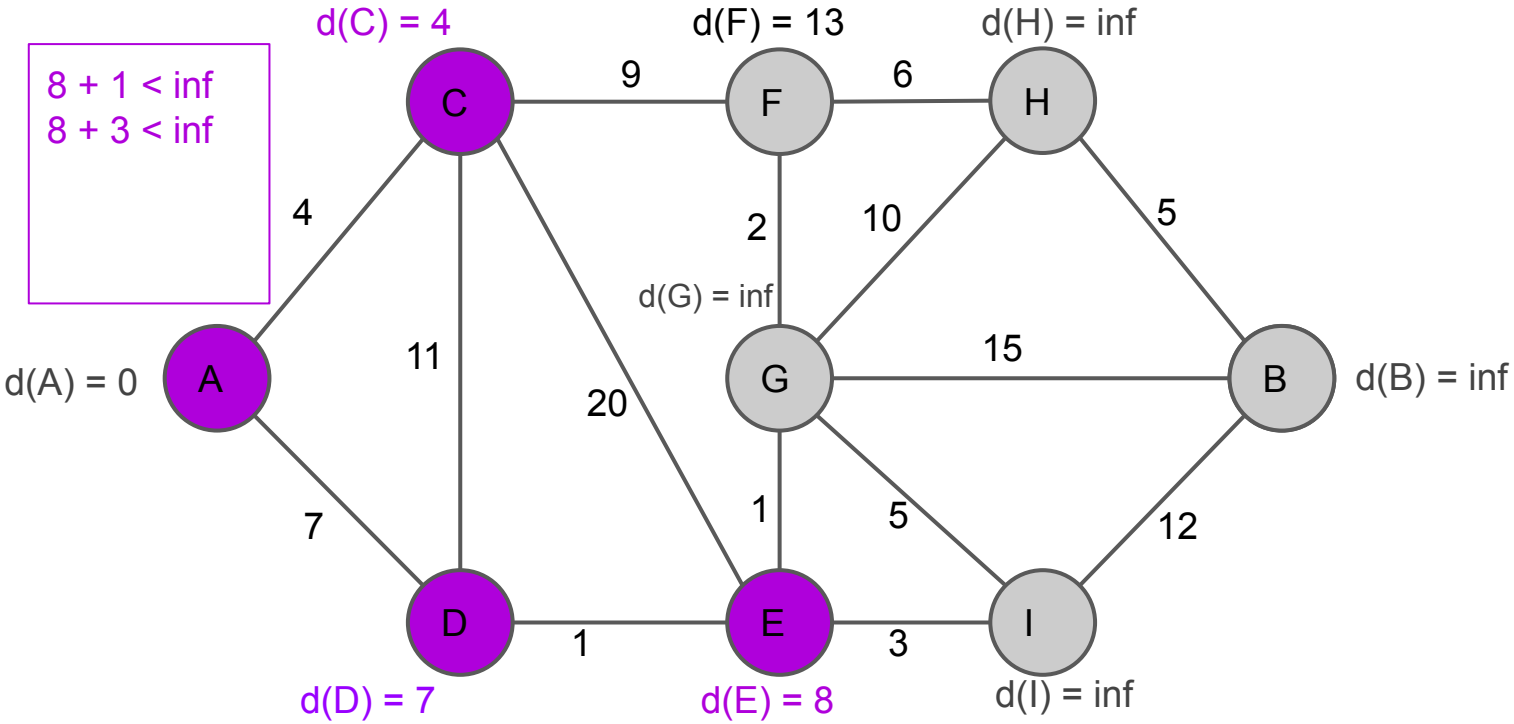
Ejemplo


d(C) = 4
d(F) = 13
d(H) = inf
7 + 1 < 24
C
9
F
6
H
4
2
10
5
d(G) = inf
11
15
d(A) = 0
A
G
B
d(B) = inf
20
1
5
12
7
D
1
E
3
I
d(D) = 7
d(E) = 24
d(I) = inf

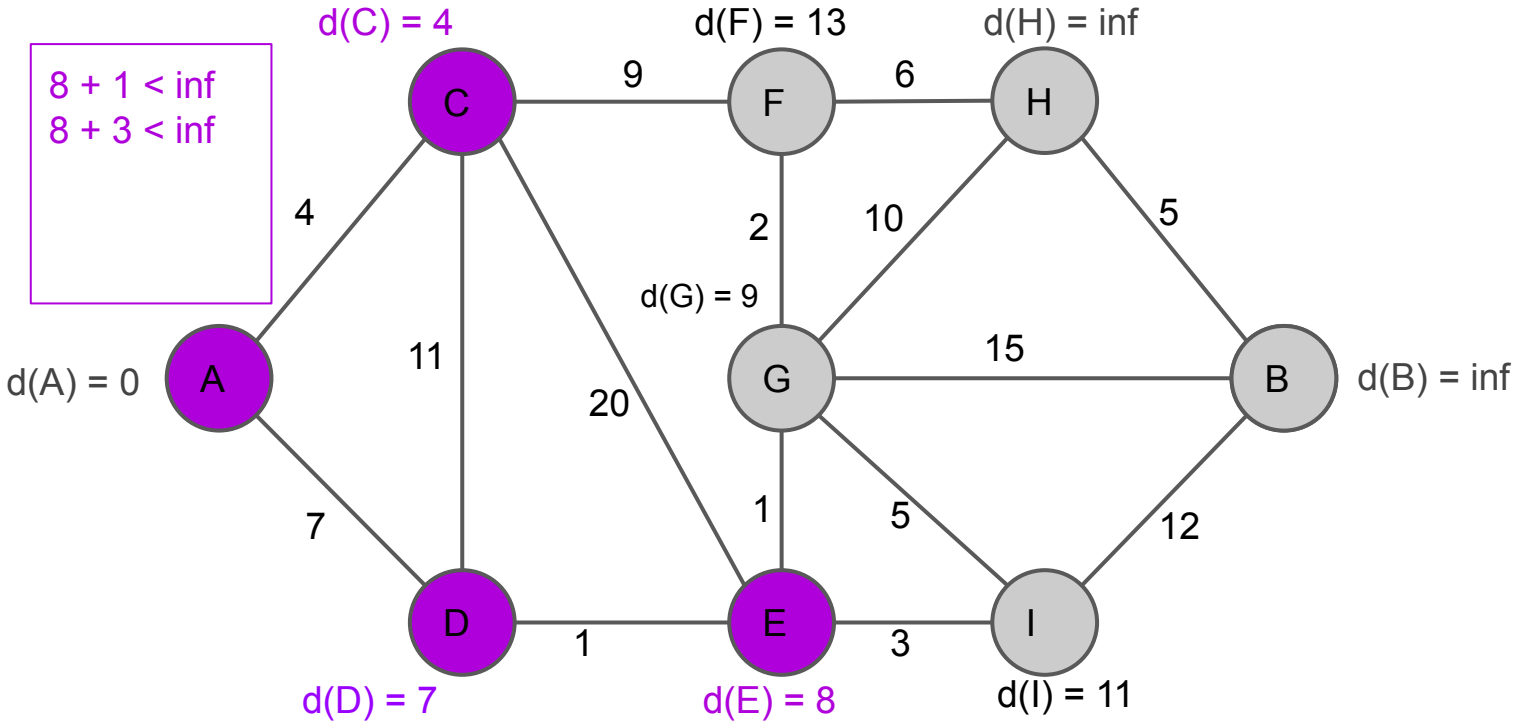
Ejemplo


Ahora visitamos el sig. nodo con menor costo
d(C) = 4
d(F) = 13
d(H) = inf
C
F
H
9
6
4
2
10
5
d(G) = inf
d(A) = 0
A
11
G
15
B
d(B) = inf
20
7
1
5
12
D
E
I
1
3
d(D) = 7
d(E) = 8
d(I) = inf

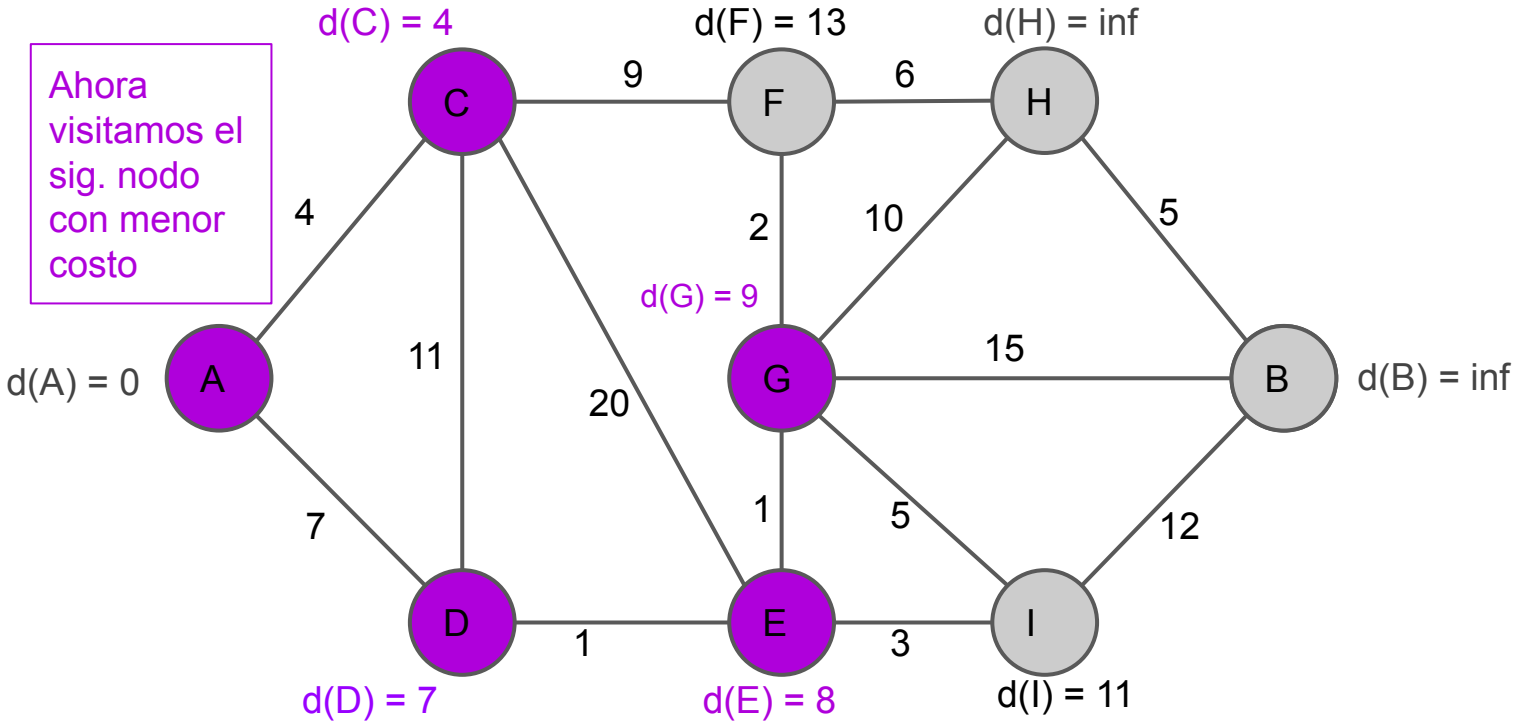
Ejemplo


8 + 1 < inf
8 + 3 < inf
d(C) = 4
d(F) = 13
d(H) = inf
d(A) = 0
d(G) = inf
d(B) = inf
d(D) = 7
d(E) = 8
d(I) = inf
A
C
D
E
F
G
H
I
B
4
11
7
9
20
1
2
10
1
6
5
15
5
12
3

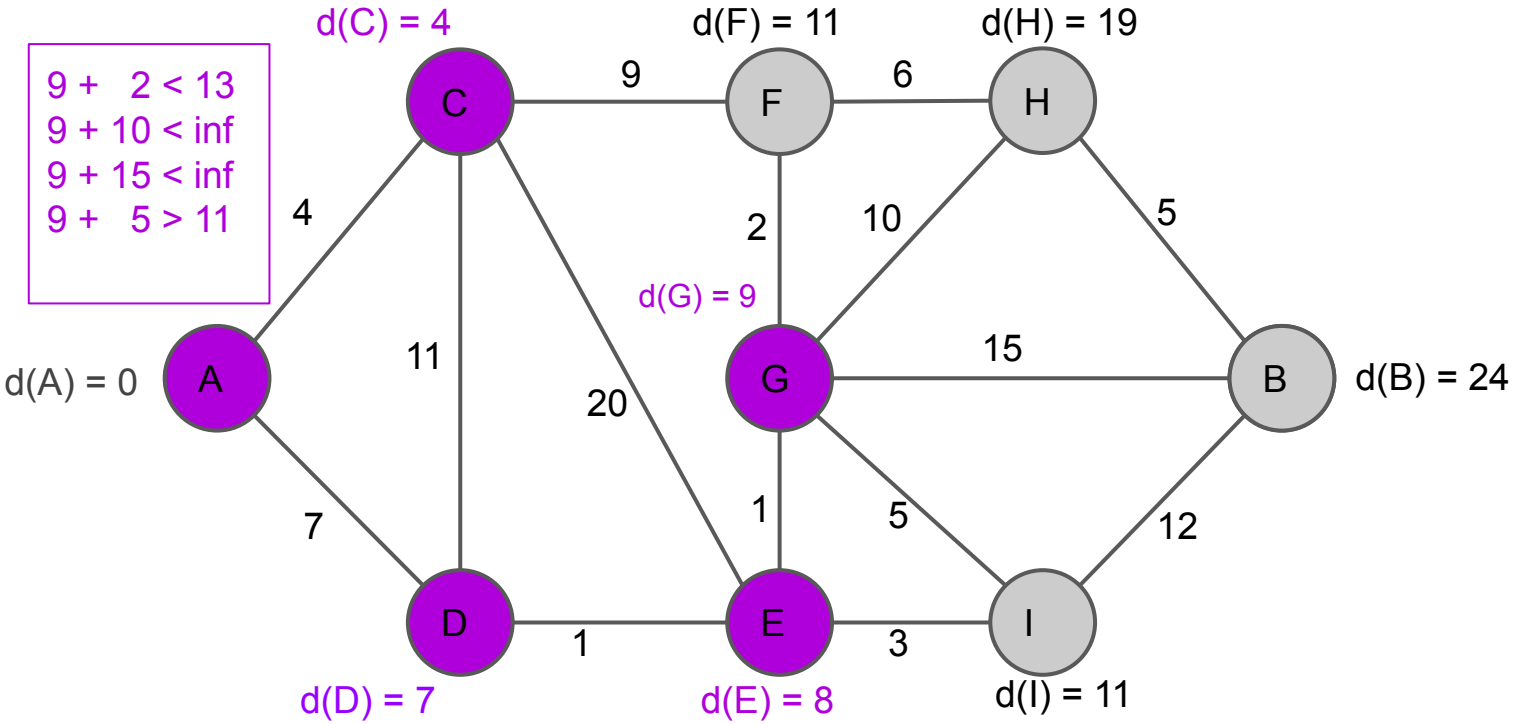
Ejemplo


d(C) = 4
d(F) = 13
d(H) = inf
8 + 1 < inf
8 + 3 < inf
C
F
H
9
6
4
2
10
5
d(G) = 9
d(A) = 0
A
11
G
15
B
d(B) = inf
20
7
1
5
12
D
E
I
1
3
d(D) = 7
d(E) = 8
d(I) = 11

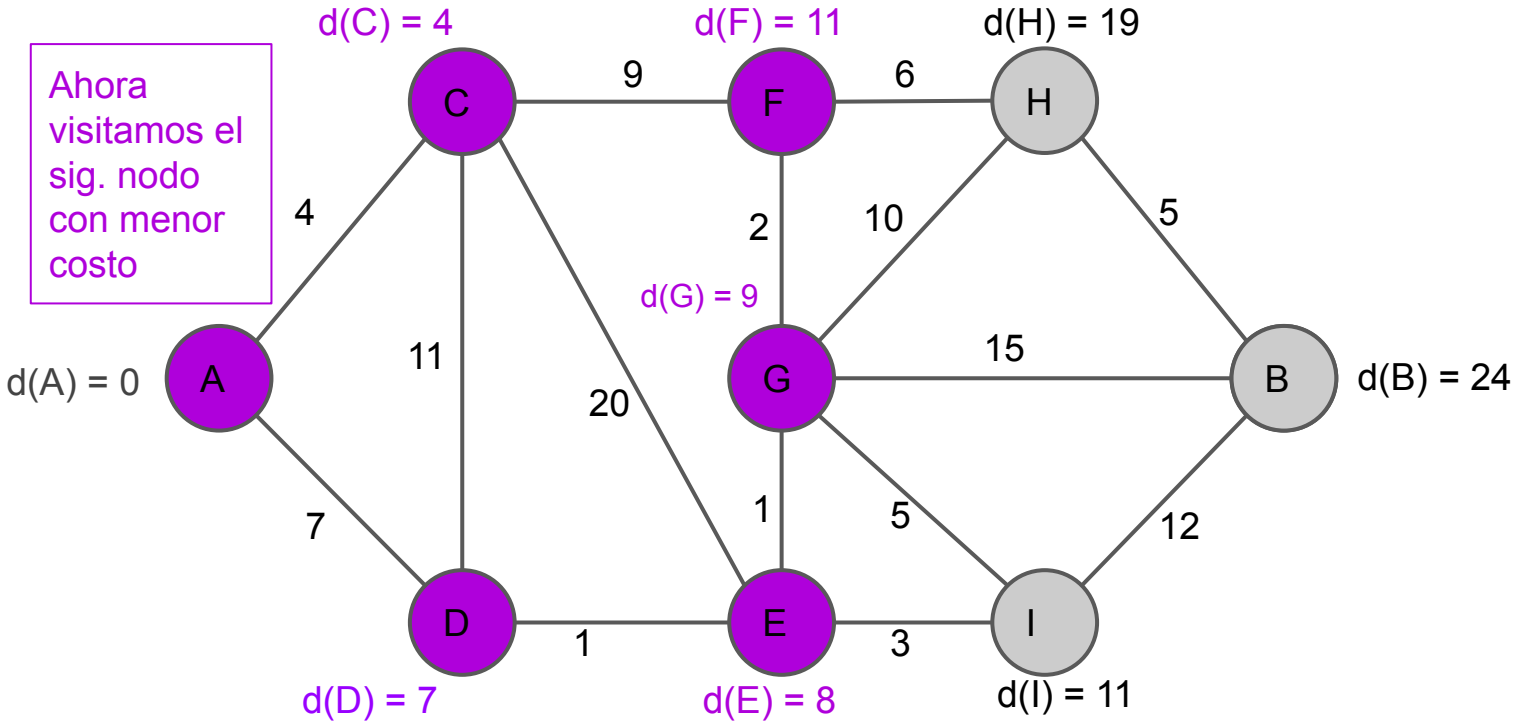
Ejemplo


Ahora visitamos el sig. nodo con menor costo
d(C) = 4
d(F) = 13
d(H) = inf
C
F
H
9
6
4
2
10
5
d(G) = 9
d(A) = 0
A
11
G
15
B
d(B) = inf
20
7
1
5
12
D
E
I
1
3
d(D) = 7
d(E) = 8
d(I) = 11

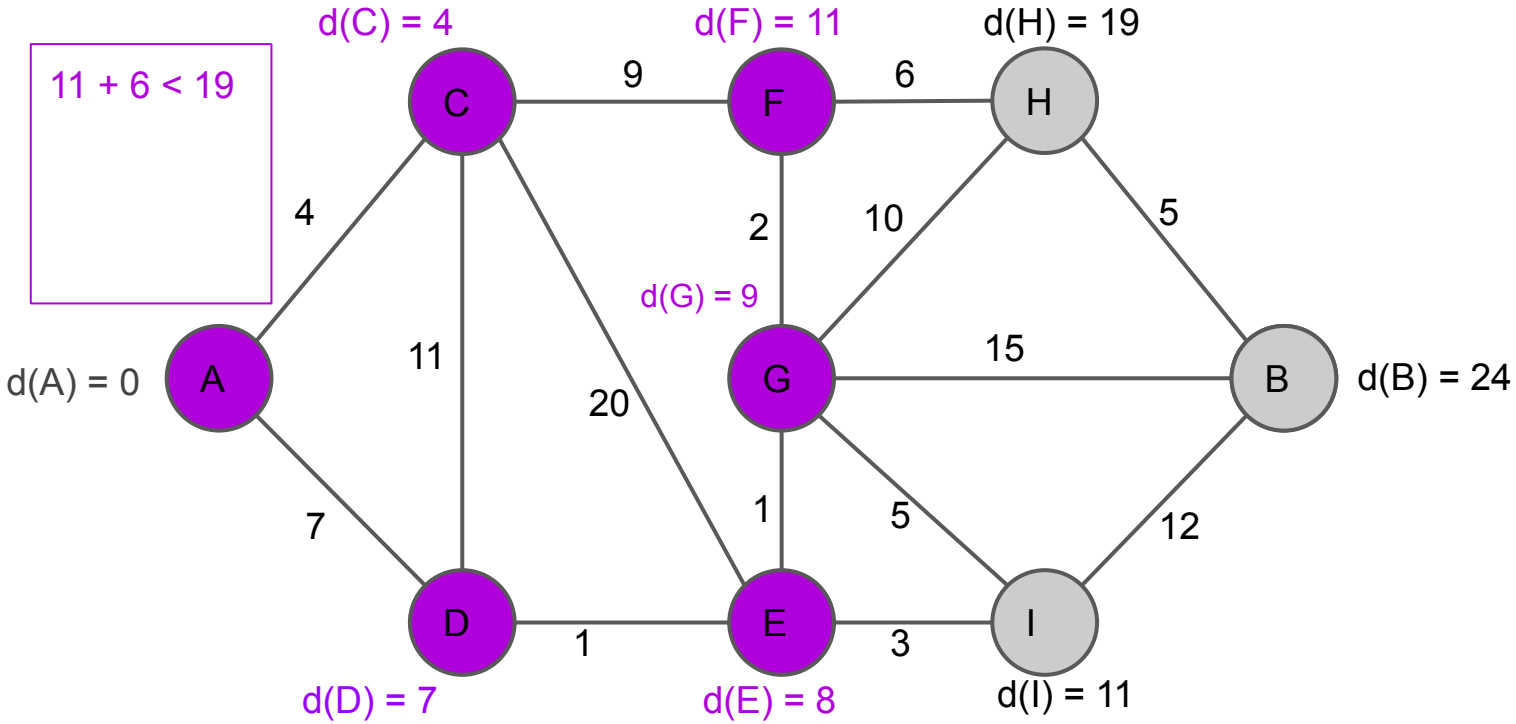
Ejemplo


9 + 2 < 13
9 + 10 < inf
9 + 15 < inf
9 + 5 > 11
d(C) = 4
d(F) = 11
d(H) = 19
C
F
H
9
6
4
2
10
5
d(G) = 9
d(A) = 0
A
11
G
15
B
d(B) = 24
20
7
1
5
12
D
E
I
1
3
d(D) = 7
d(E) = 8
d(I) = 11

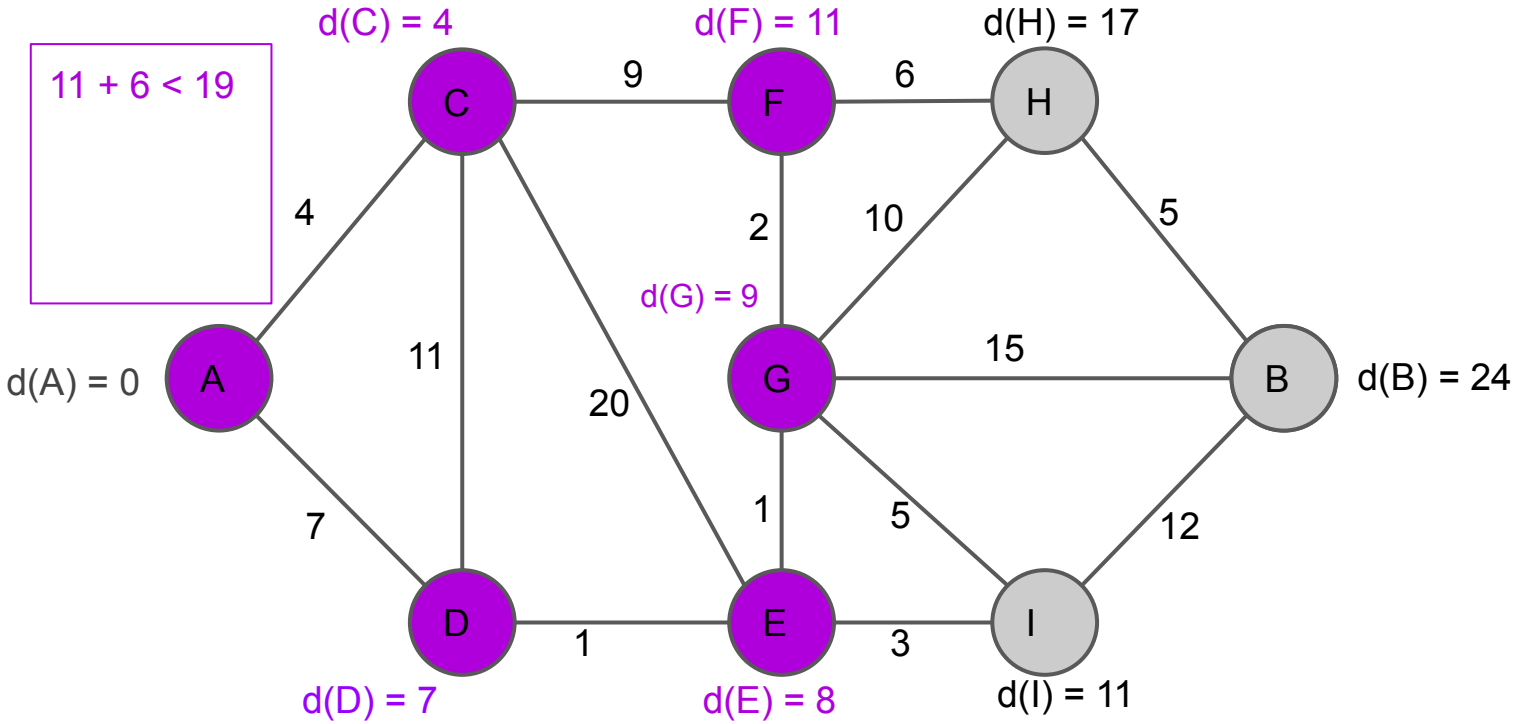
Ejemplo


Ahora visitamos el sig. nodo con menor costo
d(C) = 4
d(F) = 11
d(H) = 19
C
F
H
9
6
4
2
10
5
d(G) = 9
d(A) = 0
A
11
G
15
B
d(B) = 24
20
1
5
12
7
D
E
I
1
3
d(D) = 7
d(E) = 8
d(I) = 11

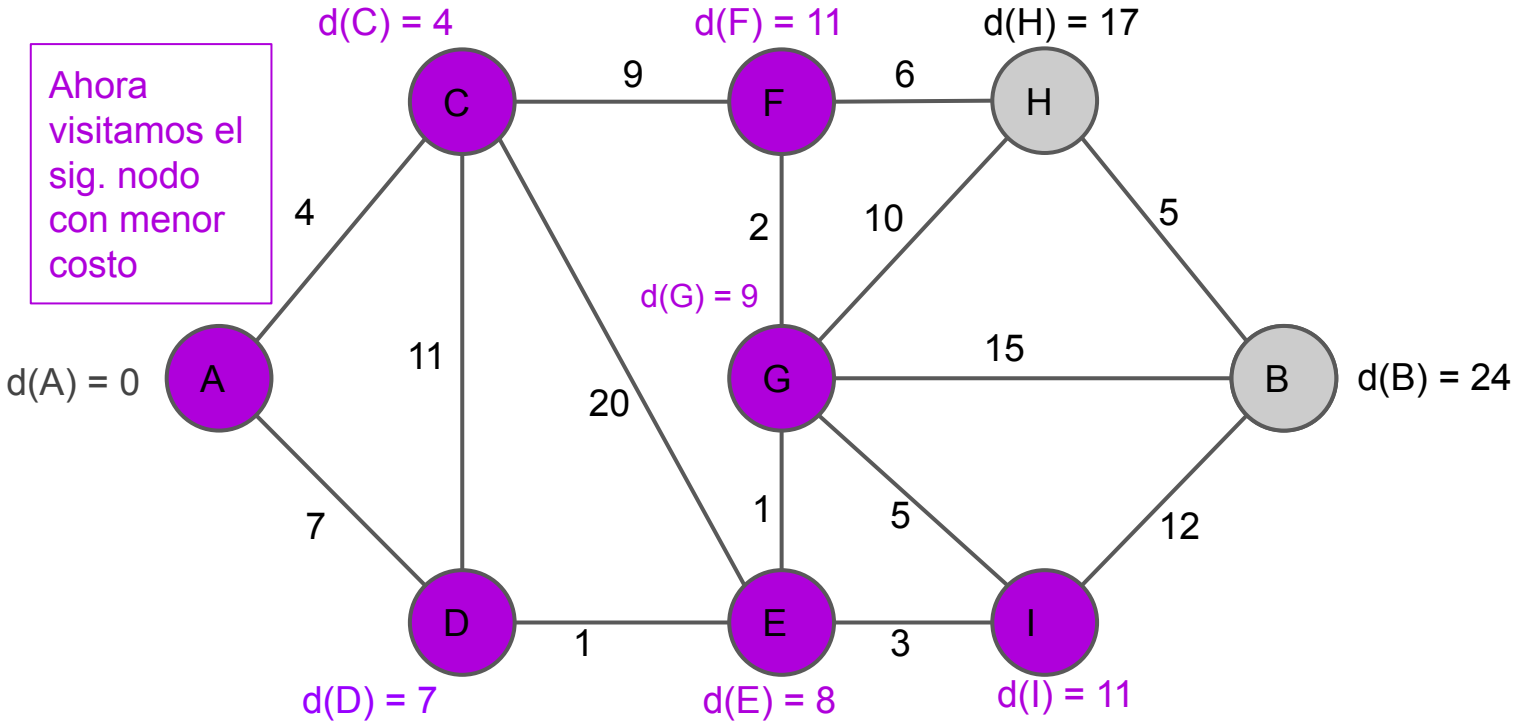
Ejemplo


11 + 6 < 19
d(C) = 4
d(F) = 11
d(H) = 19
C
F
H
9
6
4
2
10
5
d(G) = 9
d(A) = 0
A
11
G
15
B
d(B) = 24
20
7
1
5
12
D
E
I
1
3
d(D) = 7
d(E) = 8
d(I) = 11

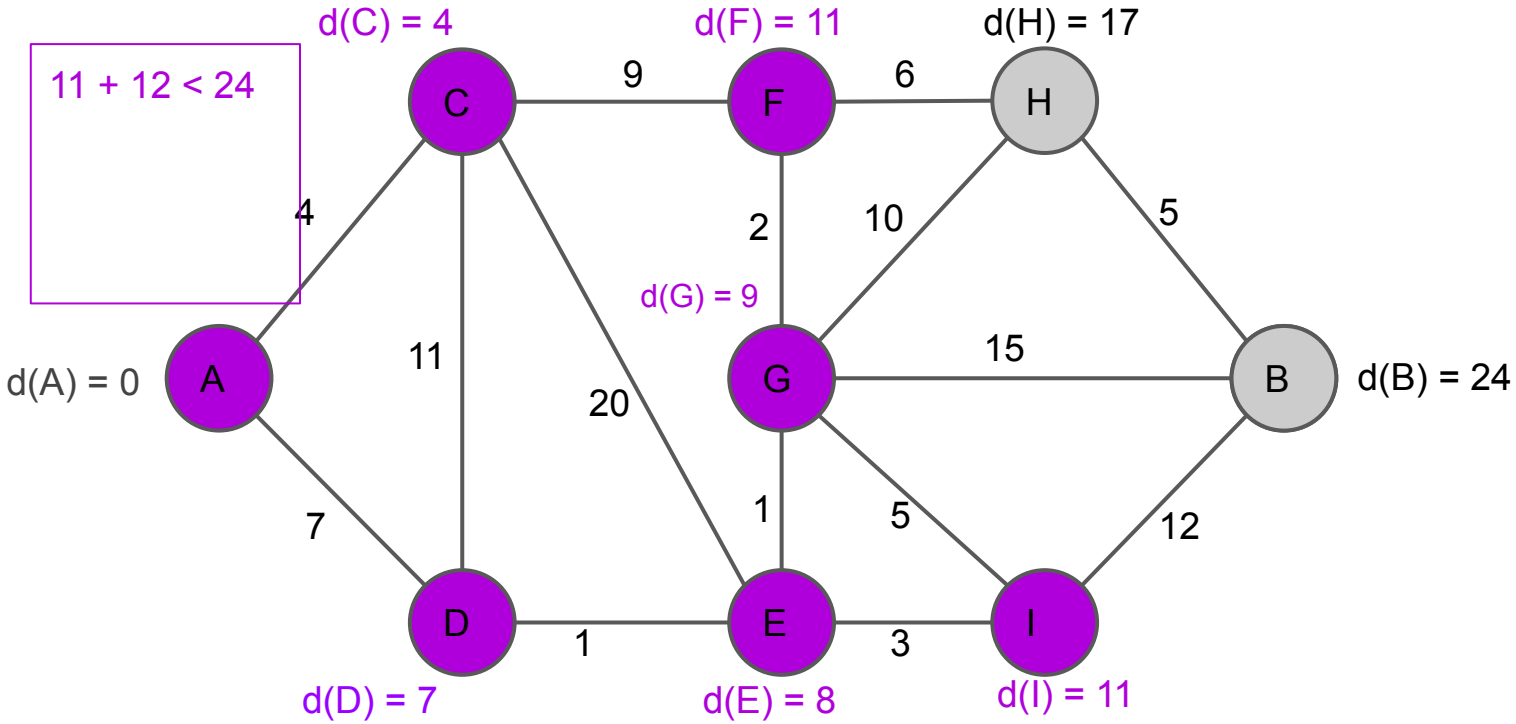
Ejemplo


11 + 6 < 19
d(C) = 4
d(F) = 11
d(H) = 17
C
F
H
9
6
4
2
10
5
d(G) = 9
d(A) = 0
A
11
G
15
B
d(B) = 24
20
7
1
5
12
D
E
I
1
3
d(D) = 7
d(E) = 8
d(I) = 11

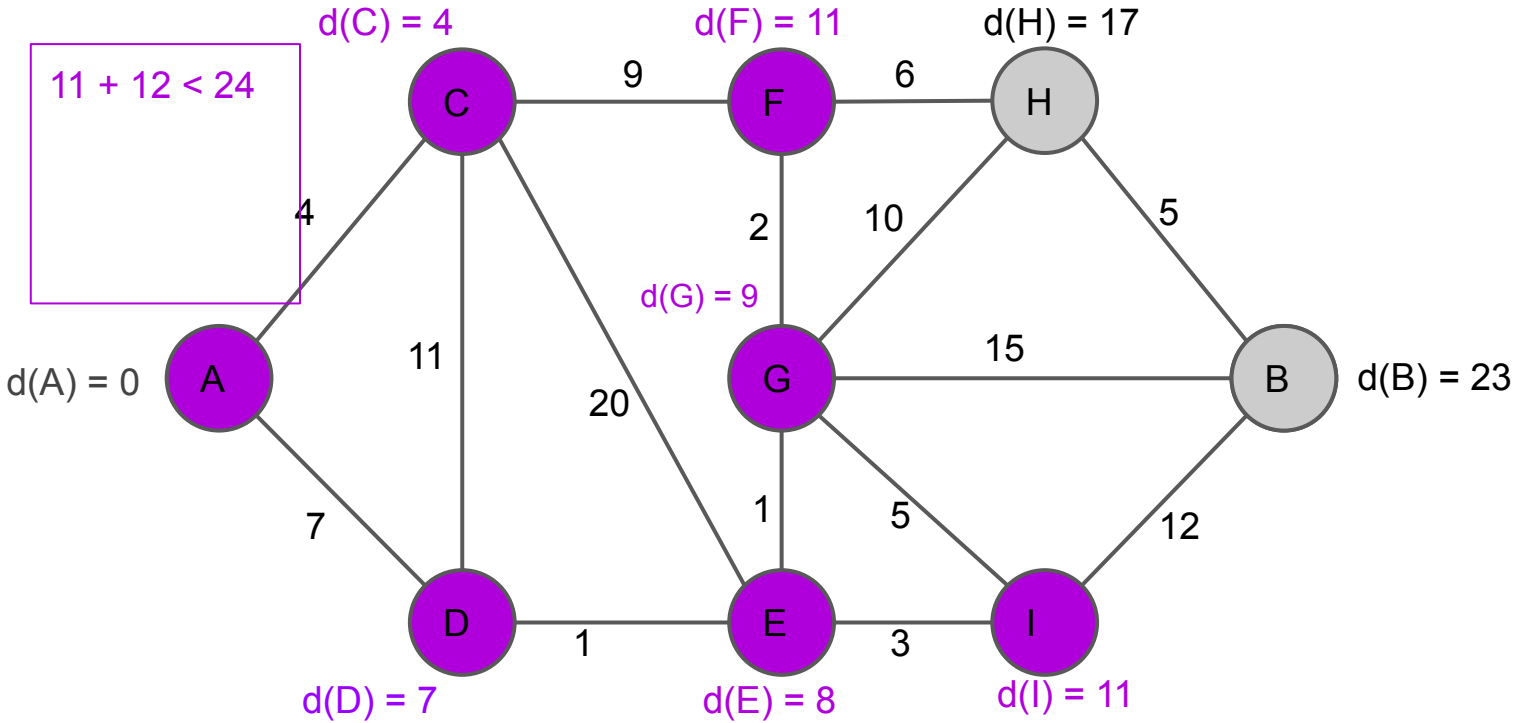
Ejemplo


Ahora visitamos el sig. nodo con menor costo
d(C) = 4
d(F) = 11
d(H) = 17
C
F
H
9
6
4
2
10
5
d(G) = 9
d(A) = 0
A
11
G
15
B
d(B) = 24
20
1
5
12
7
D
E
I
1
3
d(D) = 7
d(E) = 8
d(I) = 11

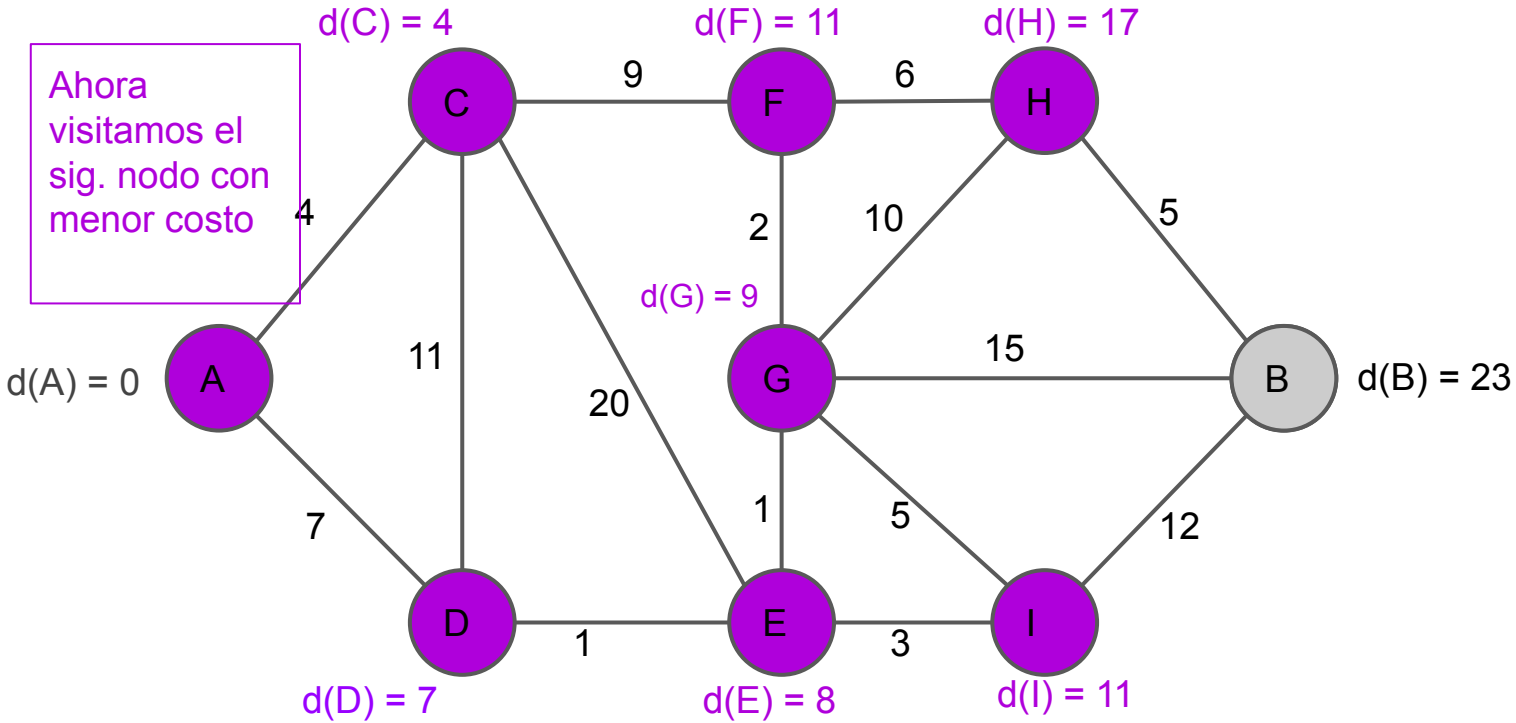
Ejemplo


11 + 12 < 24
d(C) = 4
d(F) = 11
d(H) = 17
C
F
H
9
6
4
2
10
5
d(G) = 9
d(A) = 0
A
11
G
15
B
d(B) = 24
20
7
1
5
12
D
E
I
1
3
d(D) = 7
d(E) = 8
d(I) = 11

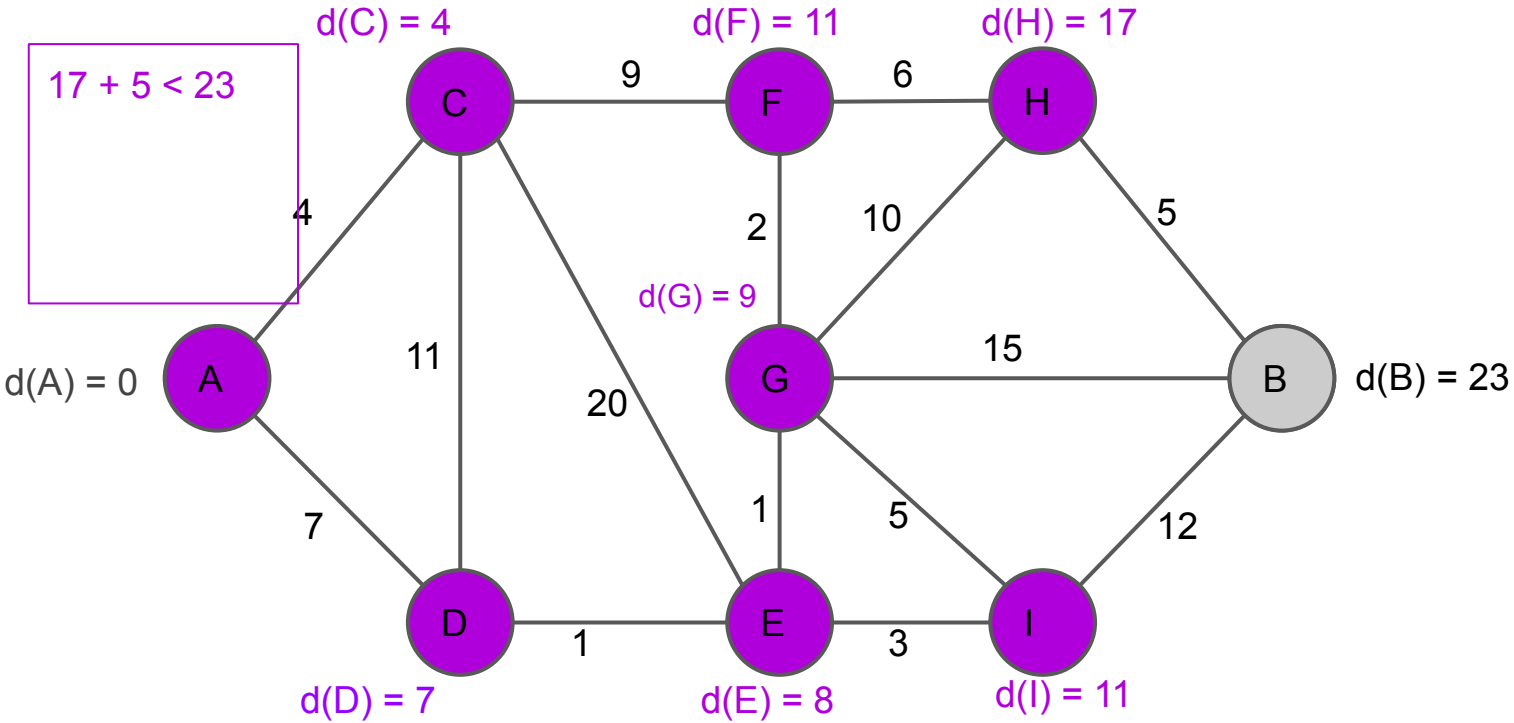
Ejemplo


d(C) = 4
d(F) = 11
d(H) = 17
11 + 12 < 24
C
9
F
6
H
4
2
10
5
d(G) = 9
d(A) = 0
A
11
G
15
B
d(B) = 23
20
7
1
5
12
D
1
E
3
I
d(D) = 7
d(E) = 8
d(I) = 11

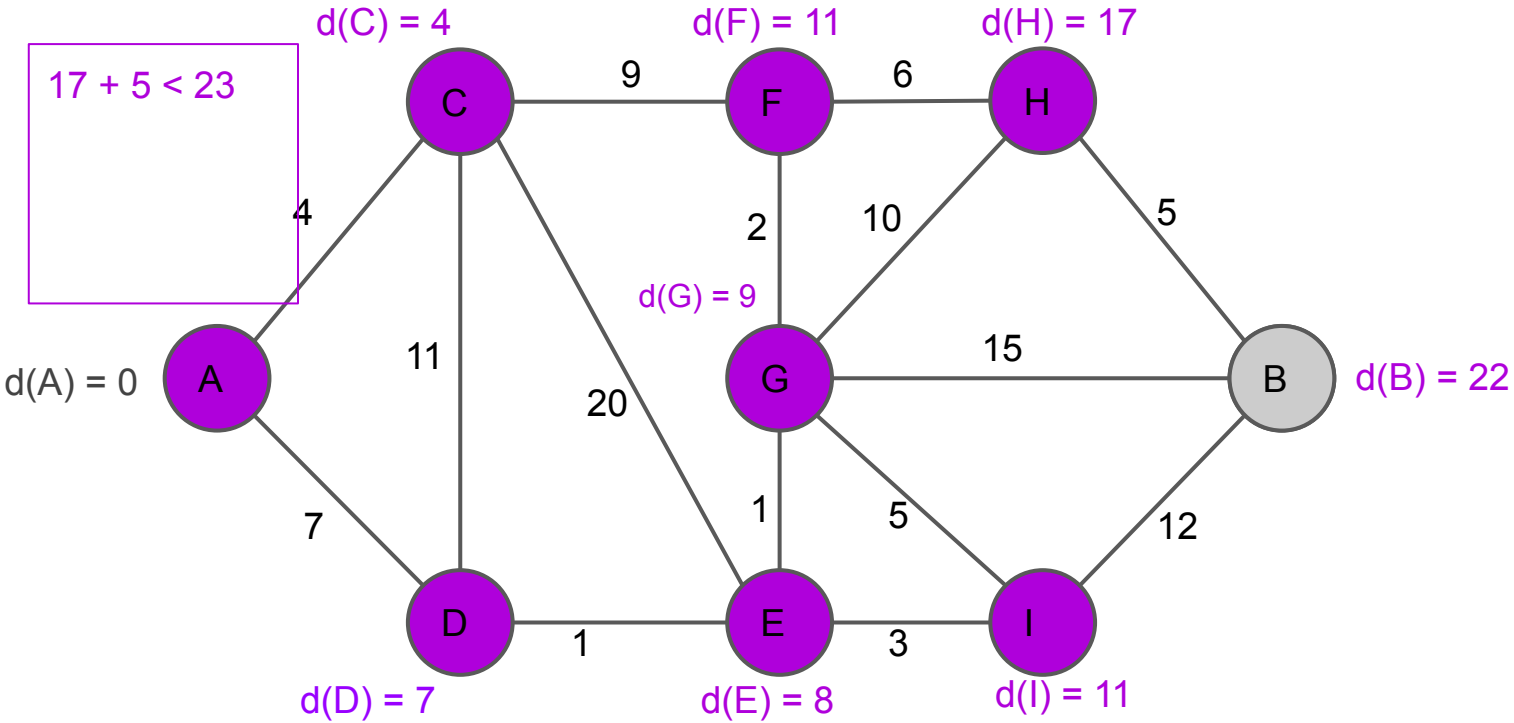
Ejemplo


Ahora visitamos el sig. nodo con menor costo
d(C) = 4
d(F) = 11
d(H) = 17
C
F
H
9
6
4
2
10
5
d(G) = 9
d(A) = 0
A
11
G
15
B
d(B) = 23
20
7
1
5
12
D
E
I
1
3
d(D) = 7
d(E) = 8
d(I) = 11

Ejemplo

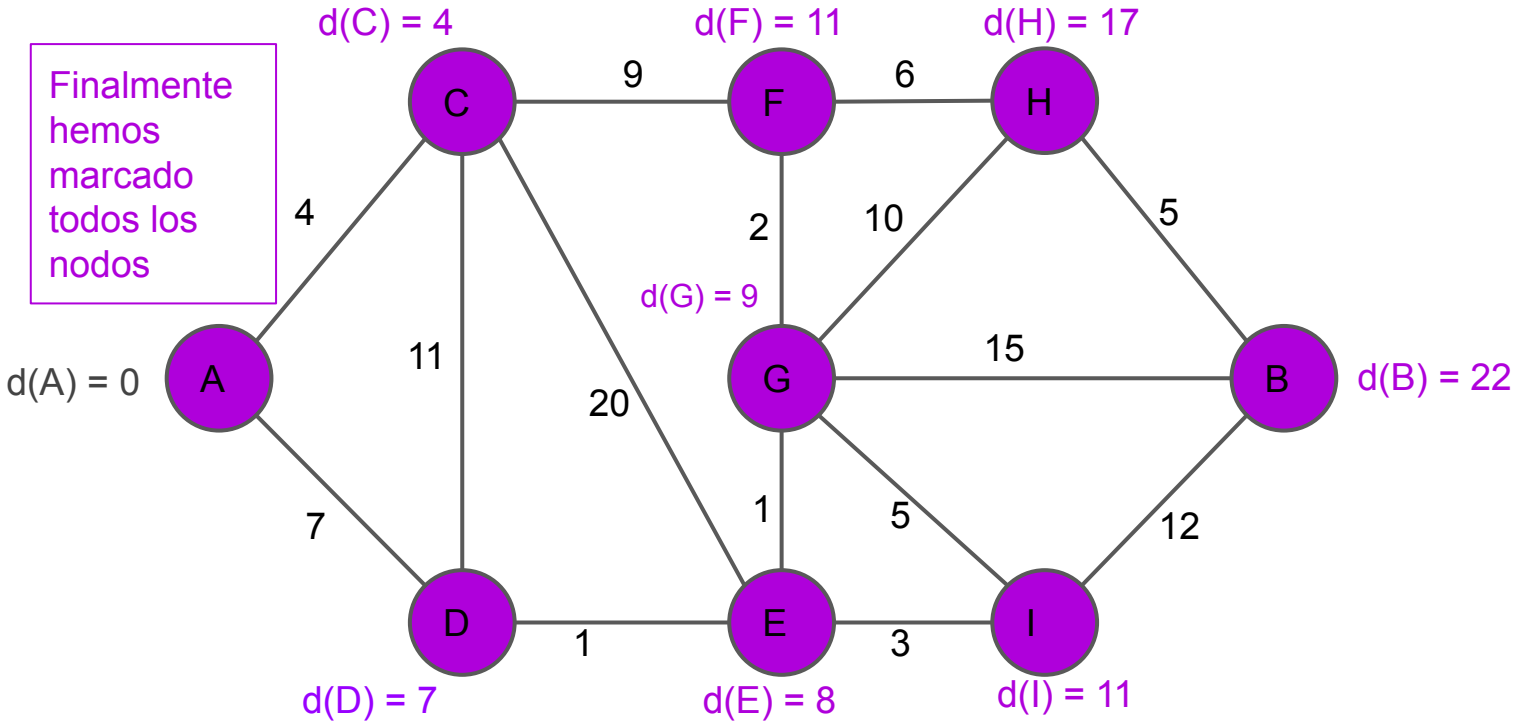

17 + 5 < 23
d(C) = 4
d(F) = 11
d(H) = 17
C
F
H
9
6
4
2
10
5
d(G) = 9
d(A) = 0
A
11
G
15
B
d(B) = 23
20
7
1
5
12
D
E
I
1
3
d(D) = 7
d(E) = 8
d(I) = 11

Ejemplo


d(C) = 4
d(F) = 11
d(H) = 17
17 + 5 < 23
C
F
H
9
6
4
2
10
5
d(G) = 9
d(A) = 0
A
11
G
15
B
d(B) = 22
20
1
5
12
7
D
E
I
1
3
d(D) = 7
d(E) = 8
d(I) = 11

Ejemplo

Finalmente hemos marcado todos los nodos



Implementación del Algoritmo

Implementación del Algoritmo

```
void Grafo<T>::dijkstra(T u){
    unordered_map<T, int> distancias;
    for (const auto& e: g_map)              // Inicializar distancias
        distancias[e.first] = INT_MAX;
    distancias[u] = 0;

    priority_queue<pair<int, T>> no_visitados;
    no_visitados.push(pair<int, T>(0, u));  // Inicializar cola de prioridad
    while (no_visitados.size() != 0){       // Mientras haya nodos sin visitar
        T v = no_visitados.top().second;
        no_visitados.pop();

        // Actualizar distancias
        for (const auto& vi: g_map[v]){
            if (distancias[vi.first] > distancias[v] + vi.second){
                distancias[vi.first] = distancias[v] + vi.second;
                no_visitados.push(pair<int, T>(-distancias[vi.first], vi.first));
            }
        }
    }
}
```

Implementación del Algoritmo

```
template <typename T>
struct Grafo{
    unordered_map<T, unordered_map<T, int>> g_map;
    void nueva_arista(T v1, T v2, int peso);
    void dijkstra(T u);
};

template <typename T>
void Grafo<T>::nueva_arista(T v1, T v2, int peso){
    g_map[v1][v2] = peso;
    g_map[v2][v1] = peso;
}
```

Implementación del Algoritmo

```
int main(){
    Grafo<char> g;
    g.nueva_arista('A','C', 4);
    g.nueva_arista('A','D', 7);
    g.nueva_arista('C','D', 11);
    g.nueva_arista('C','E', 20);
    g.nueva_arista('C','F', 9);
    g.nueva_arista('D','E', 1);
    g.nueva_arista('E','G', 1);
    g.nueva_arista('E','I', 3);
    g.nueva_arista('F','G', 2);
    g.nueva_arista('F','H', 6);
    g.nueva_arista('G','H', 10);
    g.nueva_arista('G','B', 15);
    g.nueva_arista('G','I', 5);
    g.nueva_arista('H','B', 5);
    g.nueva_arista('I','B', 12);

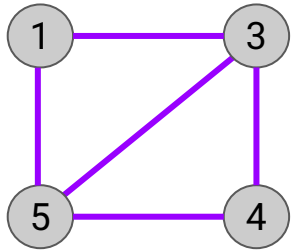
    g.dijkstra('A'); // Salida: A->0,B->22,C->4,D->7,E->8,F->11,G->9,H->17,I->11
}
```



Árboles de Expansión

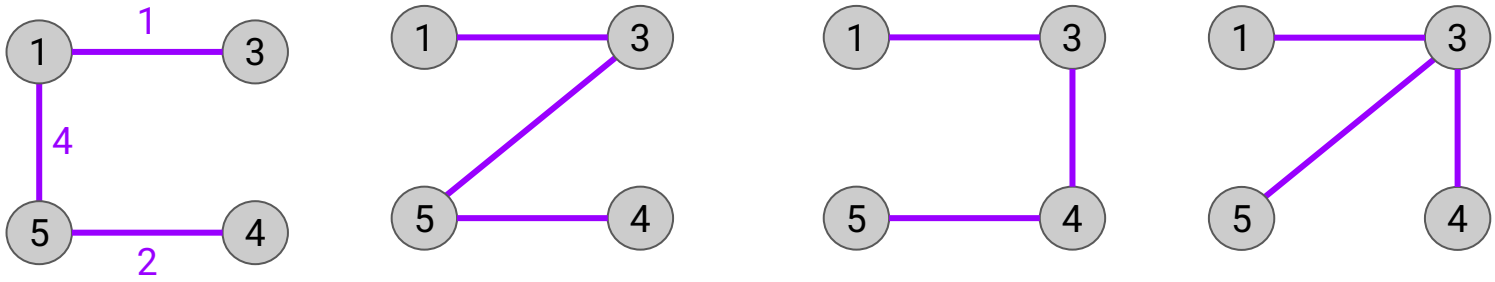
Árboles de Expansión

Dado un grafo G conexo y no dirigido. Un árbol de expansión es aquel que está compuesto por todos los vértices de G y algunas o todas las aristas de G. Este árbol debe tener una ruta entre cada par de vértices, sin bucles.



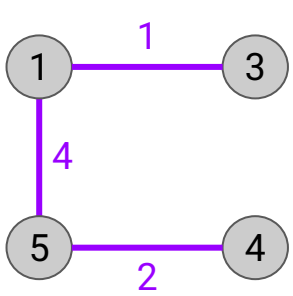
Árboles de Expansión

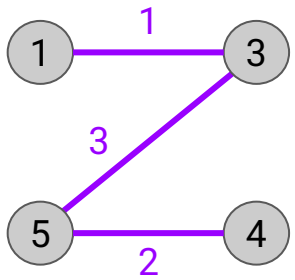
De hecho el grafo G, puede tener muchos Árboles de Expansión

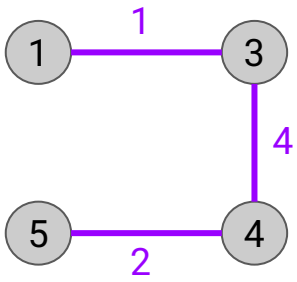


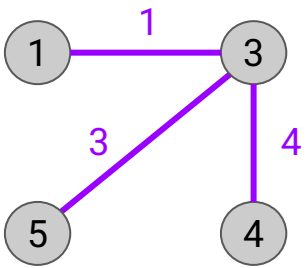
Árboles de Expansión Mínima

¿Cuál de los siguientes árboles tiene el mínimo costo/peso?



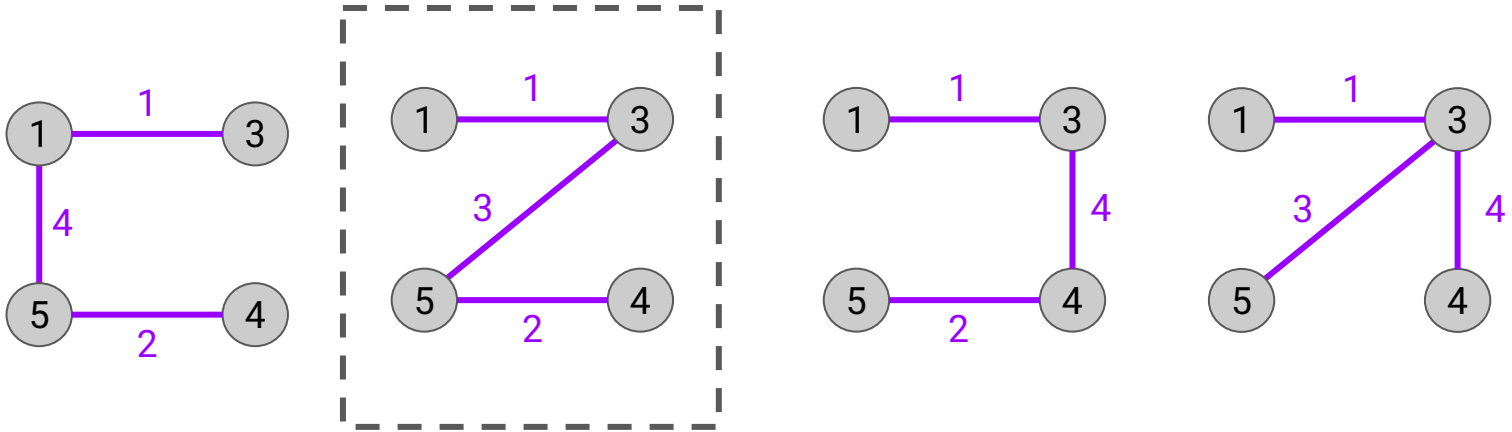






Árboles de Expansión Mínima

¿Cuál de los siguientes árboles tiene el mínimo costo/peso?



Árbol de Expansión Mínima



Algoritmo de Kruskal

Unión por Conjuntos

1. Al inicio crearemos un conjunto por cada nodo. Cada conjunto estará representado por el nodo (S_{nodo}). Por ejemplo:

$$S_A = \{A\}, S_B = \{B\}, S_c = \{C\}, \ldots$$

2. Para unir dos conjuntos, primero se debe verificar que estos no estén representados por el mismo nodo (bucle). Es decir no deben tener elementos en común para unirse (disjuntos).
3. Si dos conjuntos son disjuntos, entonces el conjunto con menos elementos será ahora representado por el nodo con el conjunto con más elementos.

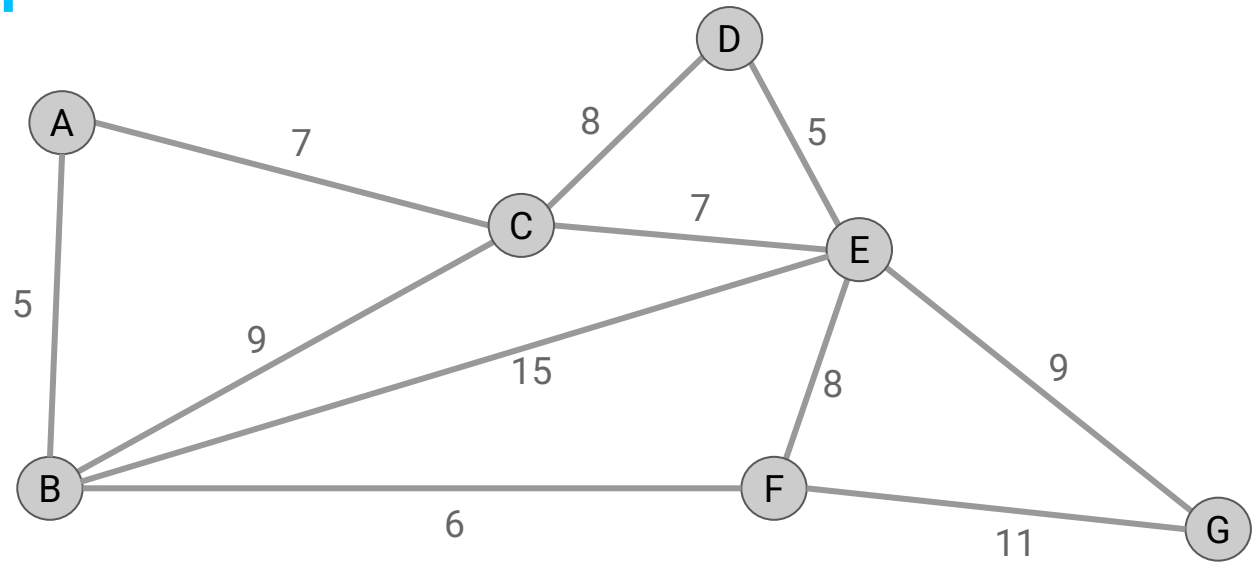
*Cardinal: El cardinal de un conjunto es el número de elementos que contiene el conjunto

Algoritmo de Kruskal

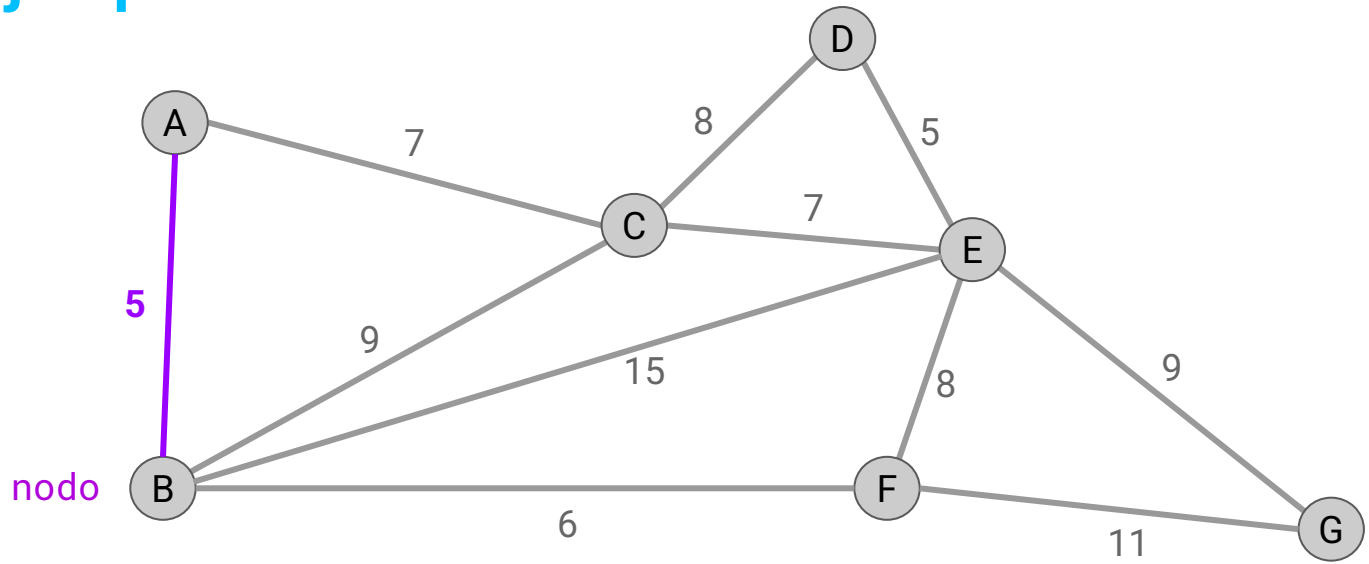
El Algoritmo de Kruskal nos permite calcular el Árbol de Expansión Mínima a través de una serie de pasos:

1. Inicializar un conjunto **`MST`** vacío para almacenar las aristas del Árbol de Expansión Mínima .
2. Ordenar las aristas por su peso, de menor a mayor.
3. Inicializar el padre de cada nodo (el mismo nodo)
4. Inicializar el cardinal para cada conjunto.
5. Para arista **`'e'`** en el conjunto de aristas ordenadas:
    a. Si los nodos de la arista no forman un ciclo
        i. Agregar la arista al **`MST`**.
        ii. Unir los conjuntos de los dos nodos **por la cantidad de elementos**.

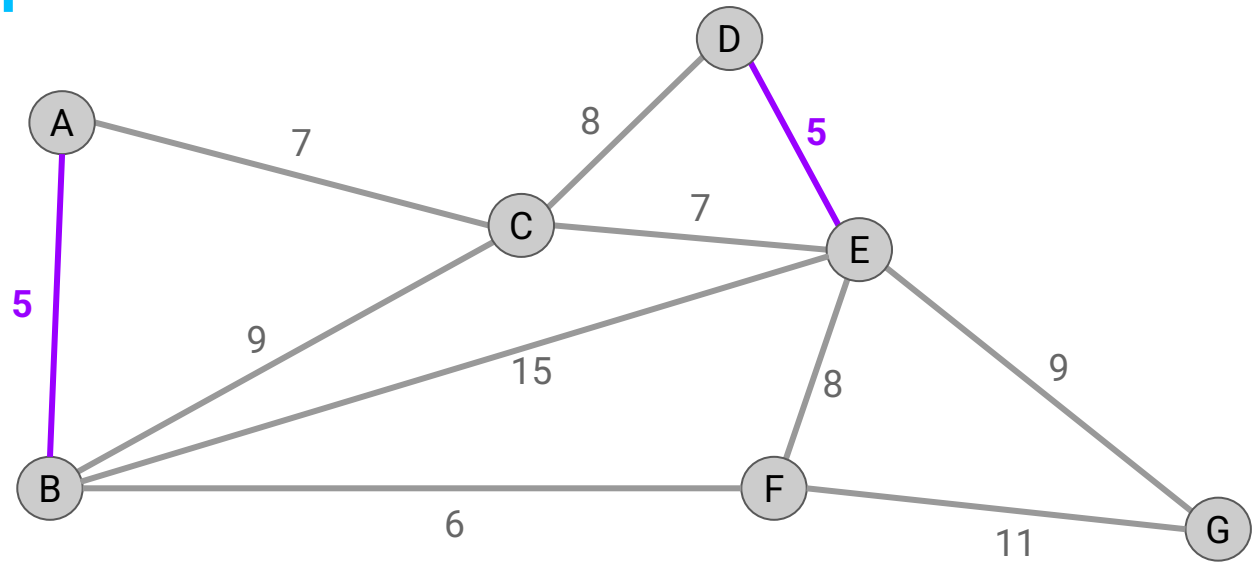
Ejemplo



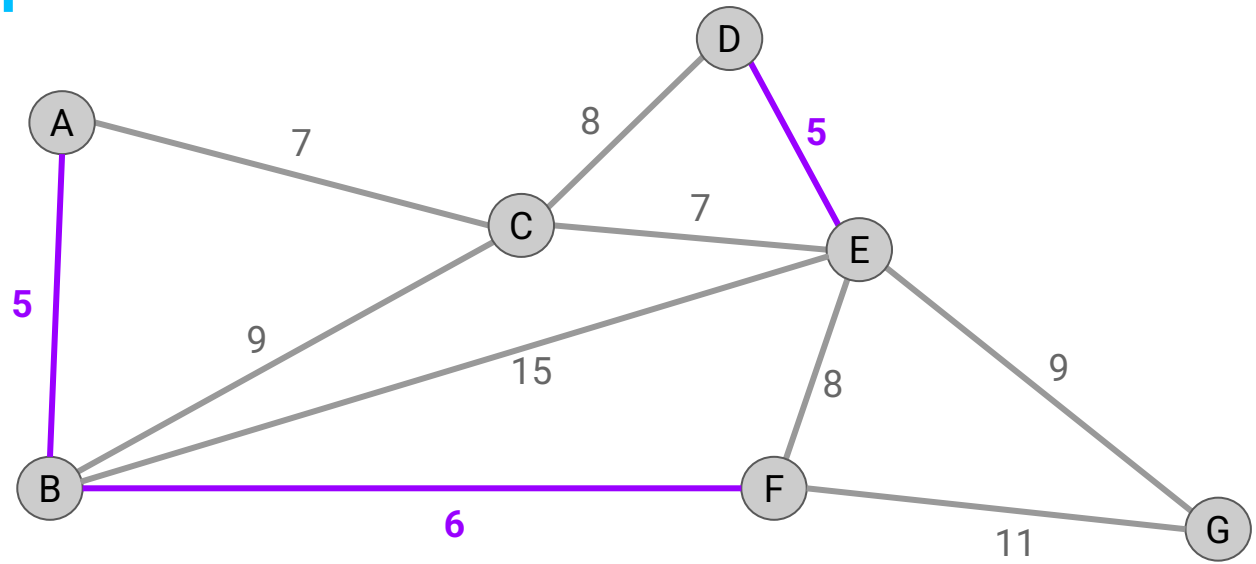
Ejemplo



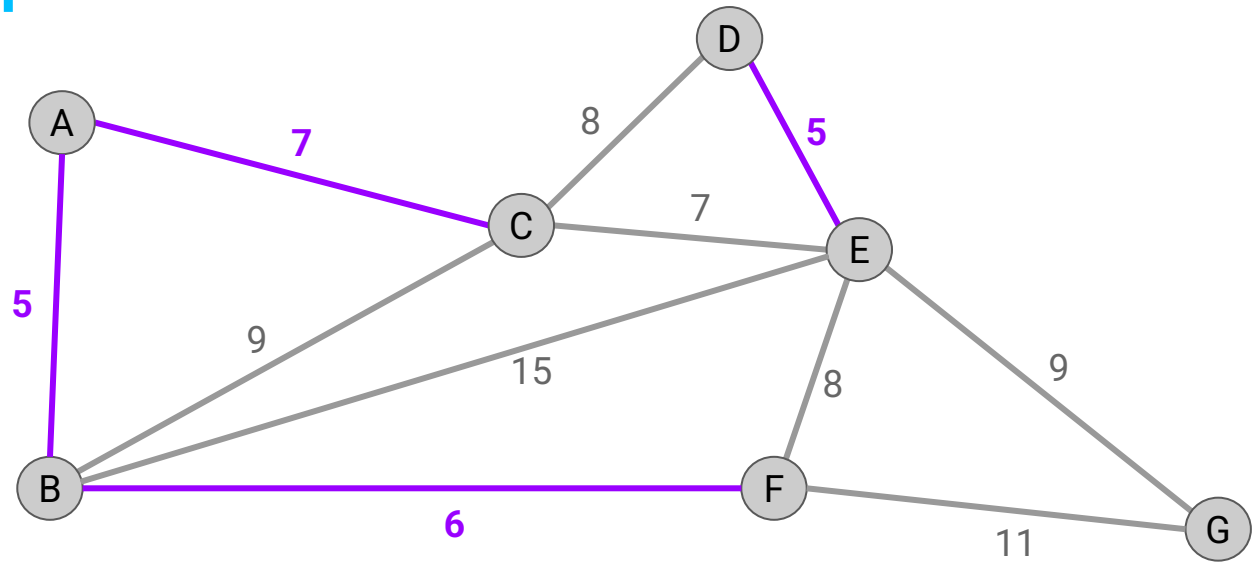
Ejemplo



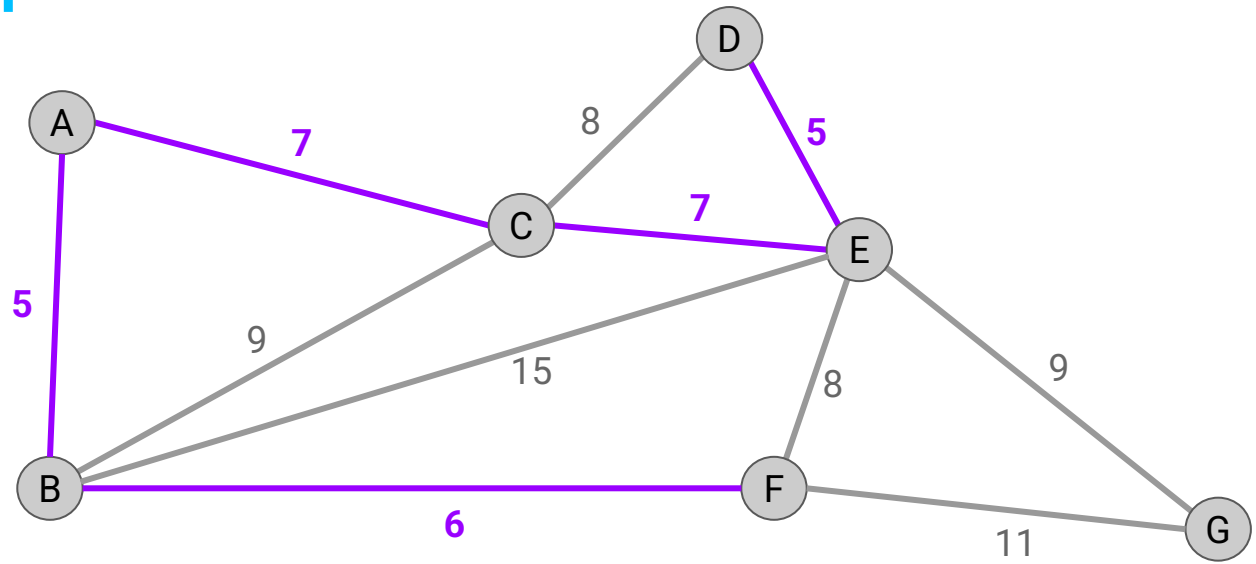
Ejemplo



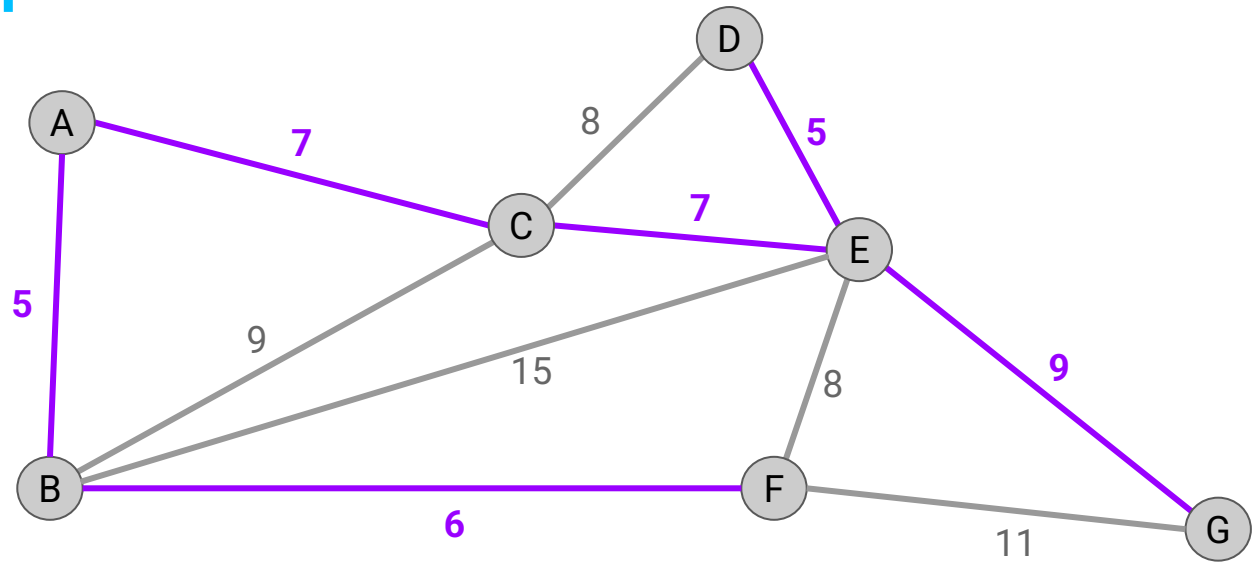
Ejemplo



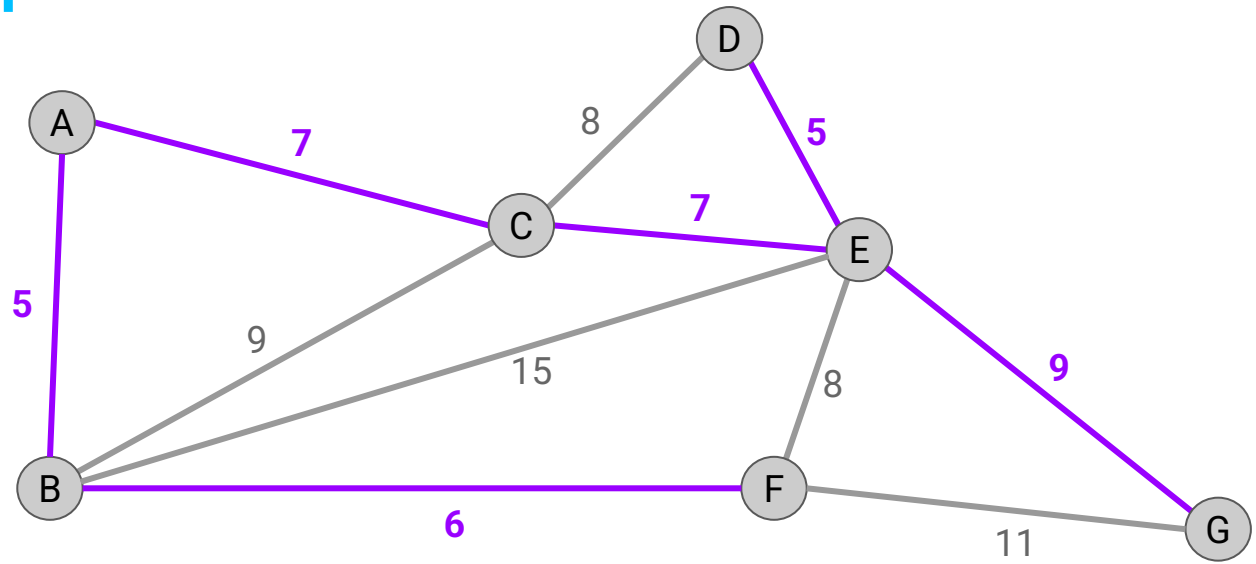
Ejemplo



Ejemplo



Ejemplo



Se han seleccionado todos los vértices (igual al número de aristas más uno)

Implementación del Algoritmo

```
void Grafo<T>::kruskal(){
    vector<arista<T>> MST;
    unordered_map<T, int> cardinal;

    sort(aristas.begin(), aristas.end()); // Ordenar aristas por peso
    for (const auto& e: aristas){
        cout << "Arista: " << e.v1 << " --- " << e.v2 << endl;
        T ancestro1 = ancestro(e.v1);
        T ancestro2 = ancestro(e.v2);

        if (ancestro1 != ancestro2){
            MST.emplace_back(e);
            // Union por conjuntos:
            if (cardinal[ancestro1] < cardinal[ancestro2])
                swap(ancestro1,ancestro2);
            padres[ancestro2] = ancestro1;
            if (cardinal[ancestro1] == cardinal[ancestro2])
                cardinal[ancestro1]++;
        }
    }
}
```

Implementación del Algoritmo

```
template <typename T>
T Grafo<T>::ancestro(T v){
    // Como el ancestro (representante) no tiene padre,
    // su padre es el mismo nodo. Significa que hemos
    // llegado al ancestro.
     if (v == padre[v])
         return v;
     else
         return ancestro(padre[v]);
}
```

Implementación del Algoritmo

```
template <typename T>
struct Grafo{
    vector<arista<T>> aristas;
    unordered_map<T, T> padres;
    void nueva_arista(T v1, T v2, int peso);
    T ancestro(T v);
    void kruskal();
    void imprimir_MST();
};
```

Implementación del Algoritmo

```
template <typename T>
void Grafo<T>::nueva_arista(T v1, T v2, int peso){
    aristas.emplace_back(arista<T>(v1, v2, peso));
    padres[v1] = v1;
    padres[v2] = v2;
    cardinal[v1] = 1;
    cardinal[v2] = 1;
}
```

Cada vez que se crea un arista, se inicializan los padres y cardinales para cada nodo

Implementación del Algoritmo

```
int main(){
    Grafo<char> g;
    g.nueva_arista('A','B', 5);
    g.nueva_arista('A','C', 7);
    g.nueva_arista('B','C', 9);
    g.nueva_arista('B','E', 15);
    g.nueva_arista('B','F', 6);
    g.nueva_arista('C','D', 8);
    g.nueva_arista('C','E', 7);
    g.nueva_arista('D','E', 5);
    g.nueva_arista('E','F', 8);
    g.nueva_arista('E','G', 9);
    g.nueva_arista('F','G', 11);
    g.kruskal();
    g.imprimir_MST();
}
```

```
Salida:

A --- B (5)
D --- E (5)
B --- F (6)
A --- C (7)
C --- E (7)
E --- G (9)
```



Resumen

En esta sesión se trataron los tópicos siguientes:

- Algoritmos Dijkstra y Kruskal